MultiFEBE (version 2.0.2)
Reference Manual

September 22, 2025

Copyright Notice and Disclaimers

Jacob D.R. Bordón
Guillermo M. Álamo
Juan J. Aznárez
Orlando Maeso

Acknowledgments

We would like to thank all the code users through the years, in particular to María Castro, María José Orjuela, Francisco González, Carlos Romero, and especially to Ángel Gabriel Vega, who is giving momentum to the project and it is the author of the tutorials. Their experience have been very valuable for debugging and developing the software. We would like to make a special mention to Luis Alberto Padrón for pushing and supporting this open source enterprise from many angles.

This software has been developed with the support of research projects:

- PID2020-120102RB-I00, funded by the Agencial Estatal de Investigación of Spain, MCIN/AEI/10.13039/501100011033.

- ProID2020010025, funded by Consejería de Economía, Conocimiento y Empleo (Agencia Canaria de la Investigación, Innovación y Sociedad de la Información) of the Gobierno de Canarias and FEDER.

- BIA2017-88770-R, funded by Subdirección General de Proyectos de Investigación of the Ministerio de Economía y Competitividad (MINECO) of Spain and FEDER.

Contents

Chapter 1

Overview

In this chapter, a brief overview of MultiFEBE download, install and usage is given. Since MultiFEBE is open source (GPLv2 license), it can also be compiled by you, see Appendix A.

1.1 Introduction

MultiFEBE is a solver for performing linear elastic static and time harmonic analyses of continuum and structural mechanics problems comprising multiple interacting regions. It is based on the Finite Element Method (FEM) and the Boundary Element Method (BEM), which are combined in order to offer many advanced features not found elsewhere. These distinctive features have been published in many scientific papers, from which [**?**, **?**, **?**, **?**, **?**] can be highlighted. No doubt, this program is heir to boundary element codes published by Professor Jose Domínguez [**?**].

Linear elastic solid materials are available for regions modeled by finite elements or boundary elements in static and time harmonic analysis, whereas linear poroelastic (Biot's theory) and inviscid fluid materials are available only for regions modeled by boundary elements in time harmonic analysis (wave propagation problems).

It has a full set of linear elastic finite elements:

- Solid (or continuum) finite elements:
  - For two-dimensional plane strain / plane stress problems:
    - 3 or 6 nodes triangular element.
    - 4, 8 or 9 nodes quadrilateral element.
- Structural finite elements:
  - 2D/3D bar element (2 nodes).
  - 2D/3D beam elements (doubly symmetric cross-section):
    - 2 and 3 nodes Euler-Bernoulli and Timoshenko straight element [**?**, **?**].
    - 2, 3 and 4 nodes curved element based on the degeneration from solid (Timoshenko theory) [**?**].
  - 3D shell elements (Reissner-Mindlin theory):
    - 3 and 6 nodes triangular shell element based on the degeneration from solid (full/selective/reduced integration) [**?**].
    - 4, 8 and 6 nodes quadrilateral shell element based on the degeneration from solid (full/selective/reduced integration) [**?**].
    - 3 and 6 nodes triangular MITC element (locking-free): MITC3i [**?**, **?**], MITC6a [**?**].
    - 4, 8 and 9 nodes quadrilateral MITC element (locking-free): MITC4 [**?**], MITC8 [**?**, **?**], MITC9 [**?**, **?**].
- Discrete finite elements:
  - 2D/3D discrete translational and translational-rotational spring-dashpot 2 node elements.
  - 2D/3D mass elements.

It also has a full set of boundary elements for linear elastic solids, poroelastic media and inviscid fluids:

- For two-dimensional plane strain / plane stress problems:
  - 2, 3 and 4 line ordinary or crack boundary elements.
  - 2, 3 and 4 line body load elements.
- For three-dimensional problems:
  - 3 and 6 triangular ordinary and crack boundary elements.
  - 4, 8 and 9 quadrilateral ordinary and crack boundary elements.
  - 2, 3 and 4 line body load elements (only in elastic regions).
  - 3 and 6 triangular body load elements.
  - 4, 8 and 9 quadrilateral body load elements.

Finite elements can be coupled to ordinary boundary, crack boundary and body load elements in order to study Soil-Structure, Fluid-Structure and Soil-Fluid-Structure Interaction in wave propagation problems. This type of mcoupling

Figure 1.1 shows an example of the modeling capabilities, where an Offshore Wind Turbine installed on a jacket structure founded on suction caissons. Finite elements model the whole foundation and structure, which includes additional masses in order to incorporate Water-Structure Interaction, and boundary and body load elements (geometrically coincident with suction caissons skirts and lids) for modeling Soil-Structure Interaction. Soil free-surface discretization is not shown for the sake of clarity.

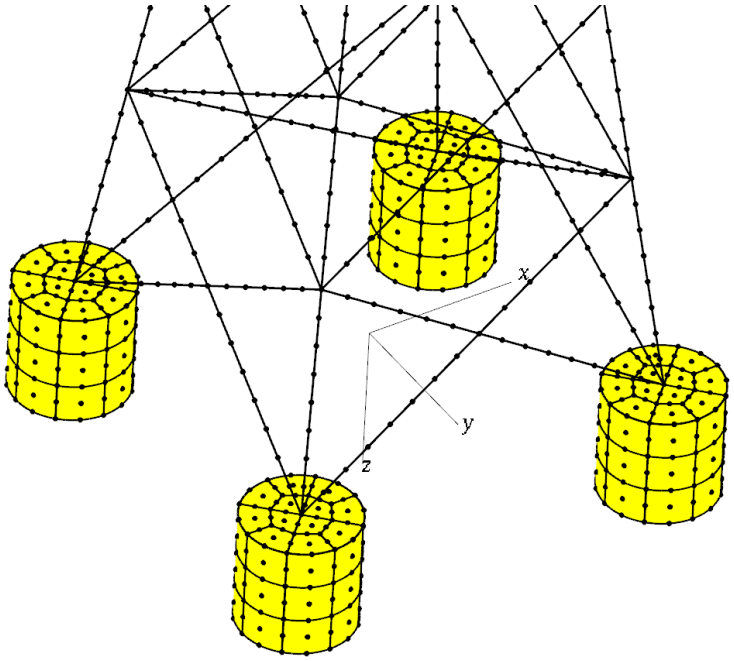


Figure 1.1: Jacket-based Offshore Wind Turbine founded on suction caissons

1.2 How to download

The **binaries** and the **source code** of MultiFEBE are available at the research group webpage and at GitHub. MultiFEBE is released under GPLv2 license. That means that you may copy, distribute and modify the software as long as you track changes/dates in source files. Also, any modifications to or software including (via compiler) GPL-licensed code must also be made available under the GPL along with build & install instructions.

MultiFEBE is available for GNU/Linux and Windows 64-bit operating systems. Binaries are released in `*.deb` and `*.sh` installer packages for GNU/Linux, and in an *.exe installer for Windows. Source code can be compiled using `cmake` + `make` + `gfortran` in GNU/Linux, and through `MSYS2/mingw-w64` + `cmake` + `make` + `gcc-fortran`. The program requires multi-core linear algebra libraries: ATLAS or OpenBLAS; which are successors of single-core LAPACK and BLAS. OpenBLAS is the default choice as it can easily be used in both GNU/Linux and Windows. The whole package is configured as a static build, i.e. the program is statically linked against libraries. Details about installing are given below.

1.3 How to install the binaries

1.3.1 GNU/Linux `*.deb` installer

The installation steps are:

1. Go to GitHub, and download the most recent `*.deb` release.
2. Make double-click on the installer.
3. Press the install button, and then type your password in the authentication window.

1.3.2 GNU/Linux `*.sh` installer

The installation steps are:

1. Go to GitHub, and download the most recent `*.sh` release (e.g. `multifebe-x.x.x-Linux.sh`).
2. Press Ctrl+Alt+T to open a terminal, and navigate to the folder where it has been downloaded.
3. Change the permissions of the file to allow its execution:

```
$ chmod +x multifebe-x.x.x-Linux.sh
```

4. Now, you can execute the installer:

```
$ ./multifebe-x.x.x-Linux.sh
```

5. The program files are decompressed by default in the same directory as the installer. The executable file is placed inside the subfolder `bin`. You may copy or move the binary `multifebe` to any folder where you want to use it, and execute as `./multifebe ...` or, if you have administrator privileges in your system, you might move the binary `multifebe` to `/usr/bin`, or create a simbolic link, so that it is available directly in the terminal as any other command.

For details about options and usage, see below section *How to use* 1.5.

1.4 How to compile the source code

The procedure to compile from the source code is explained in Appendix A.

1.5 How to use

MultiFEBE is a command-line program that you can run on a Windows terminal (cmd or PowerShell) or GNU/Linux terminal. Once installed, you can run it by executing `multifebe` from a terminal as we already mentioned at the end of the installation steps. The program execution command line is started by `multifebe` and followed by a set of arguments which tells the program on how to run (options). For instance, as we did before, if we execute:

```
$ multifebe --help
```

it will then print a brief help about how to run it and the available options:

```
$ multifebe --help
Usage:
  multifebe [options]

Options:
  -i, --input STRING        Input file name           [required]
  -o, --output STRING       Output files basename     [default: input file name]
  -m, --memory INTEGER      Max. memory allowed (GB)  [default: 0 (unlimited)]
  -b, --verbose INTEGER     Verbose level: 0-10       [default: 1]
  -v, --version             Program version
  -h, --help                This help
```

Note that each option can start with one or two hyphens, followed by respectively the short or long name of the option. In the previous case, we could request help from the program by writing either `-h` or `--help`. Some options does not require a value like `-h` or `-v`, but other options require a string or a number, which are written after each of them. Options are self-descriptive from the program help, but more details are given in Table 1.1.

Option name			
Short	**Long**	**Argument**	**Description**
`-i`	`--input`	`STRING`	`STRING` is a path to the plain text input file where the definition of the case is read. This option is thus mandatory.
`-o`	`--output`	`STRING`	`STRING` is a path to the basename of the output files (plain text) where the results of the case are printed. This option is optional since the input file path is taken by default. The output files differ only in the extension of the file.
`-m`	`--memory`	`INTEGER`	`INTEGER` is the maximum memory (in GB) allowed to be used by the program when allocating the main matrices. If the program predicts that such memory (or more) is required, then it stops itself. This option is important when dealing with large models in order to prevent the computer from memory swapping (transferring memory data to the hard-drive). Therefore, it is recommended to use this option with a setting of around 75% of the available memory of the computer.
`-b`	`--verbose`	`INTEGER`	`INTEGER` sets the level of details printed by the program when executing, being 0 the level where minimal information is printed and 10 the level where all the steps are printed.

Table 1.1: MultiFEBE command-line arguments (options)

MultiFEBE uses OpenMP in order to take full advantage of multi-core shared-memory computers. By default, the program uses all the available cores, but it is possible to control the number of cores used by previously defining the environment variable `OMP_NUM_THREADS`. Imagine you would like to only use 4 cores, then in GNU/Linux you would have to execute:

```
$ export OMP_NUM_THREADS=4
```

In Windows, it would have to execute:

```
$ set OMP_NUM_THREADS=4
```

Figure 1.2 shows a MultiFEBE usage flowchart. It operates mainly through plain text files. The program reads the input file in order to define the case (type of analysis, geometry topology, mesh, materials, boundary conditions, results to be calculated, etc). Results are written into a set of output files which depends on the type of results requested in the input file.

Most of the user effort is usually spent in preparing the geometry, mesh and then post-processing the results. In order to facilitate these tasks, we have included the possibility of reading and writing Gmsh files. Gmsh is a powerful free and open-source pre- and post-processor. The only Gmsh file format that MultiFEBE can read is MSH file format version 2.2, which is the default file format up to Gmsh 3.0.6. Newer Gmsh versions save mesh files in newer mesh file formats by default. However, you can also export meshes in older file format versions by doing `File>Export>Format: Mesh - Gmsh MSH (*.msh)>Format: Version 2 ASCII` when using the GUI, or including the option `-format msh2` when using the command-line. This integration enables the user to perform, for example, extensive parametric

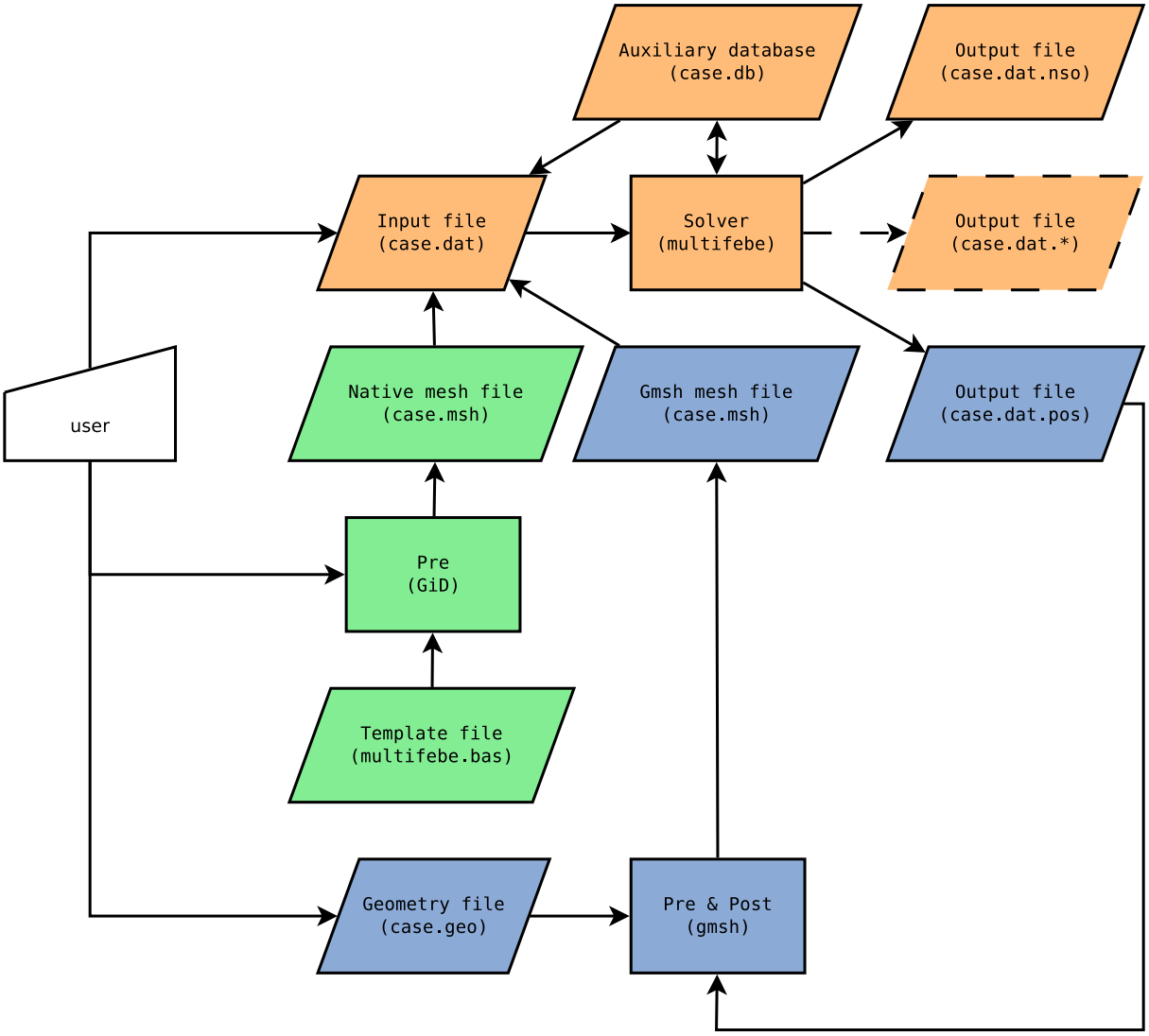


Figure 1.2: MultiFEBE usage flowchart. Orange blocks corresponds to MultiFEBE solver files and processes. Blue blocks corresponds to Gmsh pre- and post-processor files and processes. MultiFEBE is able to read Gmsh mesh files and write Gmsh postprocessing results files. Green blocks corresponds to the possibility of generating the mesh in MultiFEBE native format from GiD and a `*.bas` template file.

studies with relatively little extra effort, and effective visualization of results. We have also created a GiD `*.bas` template file which allows exporting the mesh file in the MultiFEBE native format (located in the `bin/` folder). We intend to improve the integration of MultiFEBE solver in GiD and others pre- and post-processors in the near future. In the same vein as PILEDYN, we have also created some problem-specific `https://www.mathworks.com/`MATLAB pre- and post-processors that will be available in our research group webpage.

The proposed workflow to perform an analysis is:

1. Decide the modelling approach, which will lead to the required topology and geometry. We suggest using Gmsh for creating the geometry, which allows defining the geometry parametrically through simple `.geo` files.

2. Generate the mesh with appropriate element sizes. In Gmsh, this can be easily done by defining element sizes all over the geometry using variables. Then you can set their value in order to generate a mesh. Once you have your solver results, you may have to return to this step to generate additional refined meshes in order to check results convergence.

3. Open your favourite plain text text editor, and write the case input file according to the required data, as shown in Chapter 2. If you use Gmsh for generating the mesh file, then you will have to define the path to it in section `[settings]`.

4. Run the solver. Copy the solver executable to the folder where the case input file is located. Then, open a terminal in this folder and execute:

```
$ multifebe -i case.dat
```

   where it has been assumed that the case input file is named `case.dat`. BEM models may require a huge amount of memory. We recommend that if the model is relatively big, or you are running the solver on a laptop, then you have to limit the available RAM for the program by including the following option:

```
$ multifebe -i case.dat -m 4
```

   which will stop the solver if the predicted required memory is greater that 4 GB (or any other quantity smaller than the available memory in the computer). This will prevent the computer from swapping and halting if such amount of memory is required.

5. Once the solver ends, the results are exported to one or more output files, see Chapter 3. If you used a Gmsh mesh file (`*.msh`), Gmsh post-processing files `*.pos` will be generated. If that is the case, you can visualize the results in the Gmsh GUI. Other text files containing node-by-node, element-by-element, or other results can be generated (depending on how you set the case input file). This text files can be easily processed by spreadsheet programs, MATLAB/Octave, GNU utilities such as awk, etcetera. We recommend the latter, as it is particularly fast once you learn how to use it. They can also be plotted through GNU/Linux utilities such as GNUPlot.

Chapter 2

Input file

2.1 Introduction

The input file is a **plain text** file which contains the **case data**. In the following, we will assume that the name of the input file name is `input.dat`, but any other name and extension will also be valid. The important fact is that the file must be a plain text file, and thus it must be edited as such in plain text editors such as Notepad, Notepad++ or others in Windows (but not Word or LibreOffice!), and emacs, gedit, vim or others in GNU/Linux environments.

The case data is divided into **sections**. Each section is a block of data related to an aspect of the case or the solver settings. Each section begins with a line exclusively containing a header with the section name between brackets, i.e. `[section name]`, and it ends when all the required data has been introduced or when another section header appears. It is important to note that its location within the file is not relevant. The data required for a section is defined by one or more lines after the header line. Each section has its own section format, which can be in the form of simple statements (`keyword = values`, `entity id: values`), or a sequence of numbers and strings. Only the recognized sections are read. Therefore, comments can be introduced outside the recognized ones (see example below). Recognized sections are shown in Table 2.1. Some sections are required (mandatory), some sections are required depending on the data introduced in other sections (dependant), and others are completely optional.

input.dat

```
[case description] <-- this is an unrecognized section used as a comment section
analysis of soil stresses around a 3x3 pile group

[problem]
n = 3D
type = mechanics
analysis = harmonic

[frequencies]
Hz
lin
100
0.01
10.

[symmetry planes]
plane_zx: symmetry
plane_yz: antisymmetry

...
```

All the identifiers of model entities (regions, boundaries, elements, nodes, etc) must be integers greater than 0. Floating point numbers can be expressed as `125.` (simple precision), `1.25e2` (simple precision) or `1.25d2` (double precision). Complex numbers are expressed as `(2.d3,1.d1)`.

Section name	Brief description	Character
	General sections	
`problem`	Type of problem, dimension $\mathbb{R}^n$ of the ambient space, and analysis to be performed	mandatory
`frequencies`	Frequencies for a time harmonic analysis	dependent
`settings`	Solver settings	optional
`export`	Export options and settings	optional
`bem formulation over boundaries`	Set BEM collocation strategies	optional
	Low-level entities of the model (mesh – geometric entities)	
`nodes`	Nodes of the mesh	dependant
`elements`	Elements of the mesh	dependant
`parts`	Parts of the mesh (a connected set of elements of the same intrinsic dimension)	dependant
	High-level entities of the model (physical entities)	
`materials`	Materials to be used in the model	optional
`cross sections`	Cross sections of the model. It contains data which complements the geometrical information contained in the mesh. It is required for structural elements.	dependant
`regions`	Regions (or subdomains) of the model	mandatory
`fe subregions`	Subregions of FE regions (a part with elements of intrinsic dimension $\mathbb{R}^n$ (solid) or lower (structure))	dependant
`boundaries`	Boundaries of BE regions (a part with elements of intrinsic dimension $\mathbb{R}^{n-1}$)	dependant
`be body loads`	Body loads of BE regions (a part with elements of intrinsic dimension from 0D (point) to $\mathbb{R}^n$)	dependant
	Auxiliary entities of the model	
`internal points`	Internal points of BE regions where the solution is calculated at post-processing	optional
`internal elements`	Internal elements of BE regions where the solution is calculated at post-processing	optional
`groups`	Group of entities of the same class	optional
	Model conditions	
`symmetry planes`	Symmetry planes of the model	optional
`conditions over be boundaries`	Boundary conditions of BE boundaries	dependent
`conditions over fe elements`	Boundary conditions of FE elements	optional
`conditions over nodes`	Boundary conditions of BE or FE nodes	optional
`incident waves`	Input or incident wave fields	dependant

Table 2.1: Sections of the data file

2.2 General sections

2.2.1 Section `problem`

This section defines the main aspects of the case. It is composed of one or more unordered lines containing one assignment `keyword = values`. Table 2.2 shows the recognized keywords, the values they can take, and a brief description of them.

Keyword	Requirement	Values	Description
`n`	mandatory	`2` or `2D`	Two-dimensional ambient space ($\mathbb{R}^2$)
		`3` or `3D`	Three-dimensional ambient space ($\mathbb{R}^3$)
`type`	mandatory	`laplace`	Laplace problem (unmaintained, it may not work properly)
		`mechanics`	Mechanics problem
`subtype`	dependant	`plane_strain`	Plane strain problem (only for mechanics in $\mathbb{R}^2$)
		`plane_stress`	Plane stress problem (only for mechanics in $\mathbb{R}^2$)
`analysis`	mandatory	`static`	Static analysis
		`harmonic`	Time harmonic analysis

Table 2.2: List of recognized keywords and values in section `problem`

In the following example, a static analysis of a plane strain problem is established:

input.dat

```
...

[problem]
n = 2D
type = mechanics
subtype = plane_strain
analysis = static

...
```

2.2.2 Section `frequencies`

This section is required when `analysis = harmonic` in section `[problem]`. That is because, for a time harmonic analysis, it is necessary to specify the frequencies to be analyzed, which are defined in this section. This section can be present in the input file, or in another auxiliary file whose path is defined in section `[settings]` (see section 2.2.3).

The section format is a sequence of numbers and strings. The first line is a string which indicates the units of the frequency, and it must be `Hz` (cycles per second) or `rad/s` (radians per second). The second line indicates how frequencies are defined, and it must be one of the following:

- `list` - Frequencies are defined by a given list of frequencies. The list can be unordered.
- `lin` - Frequencies are defined by a given number of uniformly (linearly) distributed frequencies between a minimum and a maximum.
- `log` - Frequencies are defined by a given number of logarithmically distributed frequencies between a minimum and a maximum.

The third line defines the number of frequencies. If the frequencies are defined by a `list`, these are defined in the fourth and the following lines (as much as the number of frequencies). If the frequencies are defined by a `lin` or `log` distribution of frequencies, the fourth and fifth lines respectively contains the minimum and maximum frequency. The general format of the section depending on how frequencies are defined (`list`, `lin` or `log`) can be illustrated as:

input.dat
```
...

[frequencies]
<units = Hz or rad/s>
list
<n = number of frequencies>
<frequency 1>
<frequency 2>
...
<frequency n>
...
```

input.dat
```
...

[frequencies]
<units = Hz or rad/s>
lin
<n = number of frequencies>
<minimum frequency>
<maximum frequency>


...
```

input.dat
```
...

[frequencies]
<units = Hz or rad/s>
log
<n = number of frequencies>
<minimum frequency>
<maximum frequency>


...
```

In the following, we are going the show three example of each way of defining the frequencies. In the first place, we show how to define an analysis at frequencies 9 Hz, 2 Hz, 5 Hz and 4 Hz. Secondly, we show how to define an analysis at 10 frequencies linearly distributed between 10 and 100 rad/s, i.e. 10 rad/s, 20 rad/s, ..., 100 rad/s. Last, we show how to define an analysis at 4 frequencies logarithmically distributed between 10 and 10000 Hz, i.e. 10 Hz, 100 Hz, 1000 Hz and 10000 Hz.

input.dat (example of list)
```
...

[frequencies]
Hz
list
4
9.
2.
5.
4.

...
```

input.dat (example of lin)
```
...

[frequencies]
rad/s
lin
10
10.
100.


...
```

input.dat (example of log)
```
...

[frequencies]
Hz
log
4
10.
10000.


...
```

2.2.3 Section `settings`

This section defines several settings of the solver. It is optional since all the parameters here defined have reasonable default values. It is composed of one or more lines containing an assignment `keyword = values`. Table 2.3 shows the recognized settings and a brief description of each one. These are related to the boundary element integration, geometrical parameters, solving of linear system of equations and auxiliary files definition.

Boundary element integration settings have an impact on accuracy and computational cost. It is not recommended to tweak these values unless you have certain specific needs. For example, if you would like to obtain very accurate results, then you can use `qsi_relative_error=1d-12`, so all quasi-singular (or nearly-singular) integrals are calculated with such accuracy. This obviously has an important impact on the computational costs related to building the linear system of equations. Singular integrals are evaluated with high accuracy by default (10^{-12}), and it cannot be adjusted by the user. On the other hand, if you would like to obtain just some fast results, you can use `qsi_relative_error=1d-3`. Note that boundary element integration accuracy is not the only source of errors, so do not expect to have such relative errors in the final results. The parameter `qsi_ns_max` defines the maximum number of subdivisions allowed when performing quasi-singular integration, but note that the quasi-singular integration strategy combines Telles's transformation and subdivision, so the default value of 16 should never be reach unless some very picky calculation is requested. The last setting `precalsets` has impact only on performance, since what it defines is the integration rules where some components of the integrand are precalculated and saved in memory.

The geometrical settings are two: the geometrical tolerance for detecting contact between geometrical entities, and and option which internally collapse the position of nodes sharing the same position within the geometrical tolerance. You should tweak these parameters when the size of the problem is very small or very large, since the geometrical tolerance is defined in absolute and not relative terms with respect to the element or mesh size.

The settings related to the linear system of equations solving allows tweaking how they are solved. You should use scaling and refining when the condition number is very large, which may happen for coupled boundary element - finite element models where regions with very different stiffnesses are connected. This obviously has some impact on the computational cost, but it is a price to pay in certain cases.

The settings related to auxiliary files allows reading some file sections from other files. In the first place, you can define the path to a **file containing the mesh**. This file can be in two different file

Keyword	Values	Default	Description
		Boundary element integration	
`qsi_relative_error`	$\in [10^{-15}, 10^{-3}]$	10^{-6}	Relative error when evaluating quasi-singular integrals (integration of boundary elements).
`qsi_ns_max`	≥ 0	16	Maximum number of subdivisions of the integration domain when evaluating quasi-singular integrals (integration of boundary elements).
`precalsets`	`n d_1 ... d_n`	`8 2 3 4 5 6 7 8 9`	Define the number `n` of pre-calculated datasets and the number `d_j` of 1D Gauss-Legendre quadrature points to be used for each dataset `j` (pre-calculation of integrand components of boundary elements).
		Geometrical parameters	
`geometric_tolerance`	> 0	10^{-6}	Geometric tolerance for detecting contact between geometrical entities. It has the same units as the mesh coordinates, i.e. no relative geometrical tolerance with respect to element/mesh size is performed.
`collapse_nodal_pos`	`T` or `F`	`T`	For each node, a ball of radius equal to the geometric tolerance and centered at the node is built, and all nodes inside this ball are moved to their center of mass
		Linear system of equations	
`lse_straight`	`T` or `F`	`T`	Perform a direct solving of the system of linear equations
`lse_scaling`	`T` or `F`	`F`	Perform scaling when solving of the system of linear equations
`lse_condition`	`T` or `F`	`F`	Estimate the condition number when solving of the system of linear equations
`lse_refine`	`T` or `F`	`F`	Refine the solution after solving of the system of linear equations
		Auxiliary files	
`mesh_file_mode`	`mode file`	`0`	Establish how the mesh is read. If `mode=0`, then the mesh is read from the input file. If `mode=1` or `mode=2`, then the mesh is read from another file specified by `file`. If `mode=1`, it is assumed that the mesh is in the native format described in the present document. If `mode=2`, then it is assumed that the mesh is in the Gmsh MSH file format version 2.2.
`frequencies_file`	`file`	“”	If defined, frequencies are read from the specified file.‘

Table 2.3: List of recognized settings and values in section `settings`

formats: native or Gmsh (MSH version 2.2). The native format is a very easy to manually generate, but it is also the file format which GiD generates after using the template file `multifebe.bas` (see Appendix C). The Gmsh mesh file format 2.2 is the default file format generated by Gmsh from Gmsh version 2.0.0 up until 3.0.6. Newer versions (Gmsh 4.0.0 onwards) generate by default a new file format version 4. However, you can still export meshes in this older file format versions by doing `File>Export>Format: Mesh - Gmsh MSH (*.msh)>Format: Version 2 ASCII` when using the GUI, or including the option `-format msh2` when using the command-line. You could also read from another file the frequencies, for example, if you save some standard set of frequencies in a file you can use in several cases, e.g. one-third octave middle frequencies for sound analysis.

Most of the times, you would only need to define the path to a mesh file and its format. For example, a Gmsh mesh file located in the same directory as the input file `input.dat`:

input.dat

```
...

[settings]
mesh_file_mode = 2 "pilegroup.msh"

...
```

2.2.4 Section `export`

This section allows the definition of export options. It is optional since all the parameters here defined have reasonable default values. It is composed of one or more lines containing one assignment `keyword = values`. It is meant to configure which and how output files have to be exported. Table 2.4 shows the recognized export options and a brief description of each of them.

User can activate/deactivate exporting general output files such as node-by-node solutions and element-by-element solutions in a native format, a file containing the wave propagation speeds of each boundary element region (particularly useful when using a Biot's poroelastic region), and a Gmsh post-processing file containing mainly all the results. More details about the output file formats are given in Chapter 3.

The user can request the calculation of kinematic and stress resultants over BE boundaries and BE body loads. That means that average kinematics (displacements and rotations) and stress resultants are calculated for each BE boundary and each BE body load present in the model. This is useful, for example, for obtaining soil reactions in soil-structure interaction problems. Note that there are to additional options to this calculation, which are defining the point where the resultant rotations and moments are calculated and if the symmetry configuration is taken into account or not.

Finally, since each file contains basically integers, real and complex numbers, and the output files are plain text files, the user can establish how these are written in the file. Files really only contain integers and reals, whose number of digits, notation, etc, can be defined via keywords `integer_format` and `real_format` respectively. Complex numbers are written as two consecutive real numbers containing whether the absolute value and argument pair (polar notation), or the real part and the imaginary part (cartesian notation).

Most of the times, you would not require to use this section. Perhaps, if you would like to directly have in your output files the complex results in cartesian notation, and you only require 8 digits after the decimal point (which reduce the file size), you could write the section as:

input.dat

```
...

[export]
real_format = eng_simple
complex_notation = cartesian

...
```

2.2.5 Section `bem formulation over boundaries`

In this section, the user can modify the BEM formulation, i.e. the type of Boundary Integral Equations (BIEs) to be used when building the BEM equations and the collocation strategy to be used. The default formulations (BIEs and collocation strategies) work for almost all cases, and any change done here should be done with care and only when necessary, see below.

There are two fundamental types of BIEs used in MultiFEBE:

Keyword	Values	Default	Description
	General output files		
`export_wsp`	`T` or `F`	`F`	Export wave propagation speeds in each boundary element region.
`export_nso`	`T` or `F`	`T`	Export nodal solutions
`nso_nodes`	`N` n_1 ... n_N	`N=-1`	Define the number `N`, and list, of nodes for which the results should be exported. If `N=-1`, results are exported for all nodes.
`export_eso`	`T` or `F`	`T`	Export element solutions
`export_pos`	`T` or `F`	-	Export results in Gmsh MSH file format version 2.2. The default behaviour is `F`, but it is automatically set to `T` if `mesh_file_mode=2` (i.e. a Gmsh mesh file is read).
	BE boundaries and BE body loads kinematic and stress resultants output file		
`export_tot`	`T` or `F`	`F`	Export the resultant average displacements and rotations, and total forces and moments of each BE boundary.
`tot_xm`	`0` or `1`	`0`	If `tot_xm=0`, then the rotations and moments are calculated with respect to the origin ($\mathbf{x}_m = \mathbf{0}$). If `tot_xm=1`, then the moments are calculated with respect to the centroid of each BE boundary.
`tot_apply_symmetry`	`T` or `F`	`T`	If set, kinematic and stress resultants are calculated taking into account the symmetry configuration.
	Export settings for writing integer, real and complex numbers.		
`real_format`	`f<W>.<d>`	`eng_double`	A Fortran-like string (descriptor) with the format for floating point numbers must be used, see Table 2.5.
	`e<W>.<d>e<E>`		
	`en<W>.<d>e<E>`		
	`sci_double` (`e25.16e3`)		
	`sci_simple` (`e16.8e2`)		
	`sci_less` (`e11.3e2`)		
	`eng_double` (`en27.16e3`)		
	`eng_simple` (`en18.8e2`)		
	`eng_less` (`en13.3e2`)		
`integer_format`	`I<W>`	`auto`	A Fortran-like string (descriptor) with the format for integer numbers must be used, see Table 2.5.
	`max` (`i11`)		
	`auto` (`<W>` = max id length)		
`complex_notation`	`polar`	`cartesian`	Complex notation to be used when exporting: polar (absolute value and argument), or cartesian (real part and imaginary part).
	`cartesian`		

Table 2.4: List of export options and values in section `export`

Variable		Format descriptor	
Integer		`Iw`	`Iw.m`
Real	Decimal	`Fw.d`	
	Exponential	`Ew.d`	`Ew.dEe`
	Scientific	`ESw.d`	`ESw.dEe`
	Engineering	`ENw.d`	`ENw.dEe`

`w`: the number of positions to be used

`m`: the minimum number of positions to be used

`d`: the number of digits to the right of the decimal point

`e`: the number of digits in the exponent part

Table 2.5: Fortran descriptors

Singular BIE (SBIE) It is also called the displacement or pressure BIE, since this equation relates the primary field variable (displacements in elasticity, pressure in acoustics, ...) at the collocation point with the the primary and secondary (tractions in elasticity, fluxes, ...) field variables over the boundaries and the body loads. This BIE can be collocated everywhere (the collocation point is the location of the Dirac delta of the Green's function). However, the nodal collocation (collocation at the boundary element nodes) is the ideal since it leads to the best conditioning of the system. Non-nodal collocation is also possible, but it should be used only where nodal collocation turns problematic.

Hypersingular BIE (HBIE) It is also called the traction or flux BIE, since it relates the secondary field variable with the the primary and secondary field variables over the boundaries and body loads. This BIE can be collocated only where the discretization has $\mathcal{C}^1$ displacement continuity. Since MultiFEBE only considers $\mathcal{C}^0$ continuous elements, this means that HBIE must have non-nodal collocation (except at element interior nodes).

The non-nodal collocation strategy implemented here is the so-called Multiple Collocation Approach[1] (MCA). When building the BIE associated with a given node, the MCA consists on building the equation by adding up as many BIEs as the number of elements sharing the node, being each BIE collocated near the node but shifted towards inside the element. For a given element, the local coordinates of the collocation point (ξ_k^i, η_k^i) contributing to the node k located at (ξ_k, η_k) is controlled by the dimensionless distance δ, which is a measure of the separation between the collocation point and the corresponding node such that:

$$\text{Line elements:} \quad \xi_k^i = (1-\delta)\,\xi_k,\ \xi_k \in [-1,1] \tag{2.1}$$

$$\text{Quadrilateral elements:} \quad \xi_k^i = (1-\delta)\,\xi_k,\ \xi_k \in [-1,1] \tag{2.2}$$

$$\eta_k^i = (1-\delta)\,\eta_k,\ \eta_k \in [-1,1] \tag{2.3}$$

$$\text{Triangular elements:} \quad \xi_k^i = (1-\delta)\,\xi_k + \delta/3,\ \xi_k \in [0,1] \tag{2.4}$$

$$\eta_k^i = (1-\delta)\,\eta_k + \delta/3,\ \eta_k \in [0,1]\,,\ \xi_k + \eta_k \leq 1 \tag{2.5}$$

This way $\delta \to 0$ means that the collocation point is approaching the node, and $\delta \to 1$ means that the collocation point is approaching the element's centroid in local coordinates. Formally $0 < \delta < 1$, but in practice it should be $0.01 < \delta < 0.6$. By default, we use is $\delta = 0.42$ for linear elements, $\delta = 0.23$ for quadratic elements and $\delta = 0.14$ for cubic elements.
The general format for this data section is:

general format of section [bem formulation over boundaries]

```
[bem formulation over boundaries]
boundary <boundary id>: <keyword: type of formulation> <values>
```

where the available formulations and their options are given in Table 2.6.

For instance, in a problem with six boundaries, where the collocation strategy in all boundaries must be nodal, with the exception of boundary 6, where a non-nodal collocation strategy is prefered for all the nodes along its boundaries, with a displacement towards inside each element of 10% the width of the element, one should write:

[1] J. Domínguez & M.P. Ariza. A direct traction BIE for three-dimensional crack problems. Engng Anal Bound Elem 2000;24:727–38.

Keyword	Values	Description
		Ordinary boundaries
`sbie`	none	SBIE with nodal collocation strategy for all nodes of the boundary. This is unsafe since at doubled nodes, depending on the boundary conditions, it can lead to a singular linear system of equations. This option can not be used in truncated boundaries, such as those for surfaces of unbounded regions.
`sbie_boundary_mca`	δ	**This is the default formulation for ordinary boundaries.** SBIE with nodal collocation strategy for all nodes of the boundary except for the nodes at the boundary of the boundary, where MCA is used. The default value is $\delta = 0.05$.
`sbie_mca`	δ	SBIE with MCA for all nodes of the boundary. The default value is indicated by Note 1.
`hbie`	δ	HBIE with MCA for all nodes. For using default values (see Note 1), the user must write a value $\delta \leq 0$. Other values
`dual_bm_same`	δ	Burton and Miller formulation (SBIE+i/kHBIE) for eliminating spurious frequencies when solving some exterior problems. SBIE and HBIE with MCA (same δ) for all nodes. The default value is indicated by Note 1.
`dual_bm_mixed`	δ_S δ_H	Burton and Miller formulation (SBIE+i/kHBIE) for eliminating spurious frequencies when solving some exterior problems. SBIE with nodal collocation for all nodes of the boundary except for the nodes at the boundary of the boundary where MCA is used (δ_S), and HBIE with MCA (δ_H) for all nodes. For SBIE, the default value $\delta_S = 0.05$. For HBIE, the default value δ_H is indicated by Note 1.
		Crack-like boundaries
`same`	δ	**This is the default formulation for crack-like boundaries.** Dual BEM (SBIE/HBIE) formulation for crack boundary elements with MCA for all nodes. The default value δ is indicated by Note 1.
`mixed`	δ_S δ_H	Dual BEM (SBIE/HBIE) formulation for crack boundary elements. SBIE with nodal collocation for all nodes of the boundary except for the nodes at the boundary of the boundary where MCA is used (δ_S), and HBIE with MCA (δ_H) for all nodes. For SBIE, the default value $\delta_S = 0.05$. For HBIE, the default value δ_H is indicated by Note 1.

- General note 1: Formally $0 < \delta < 1$, but in practice it should be $0.01 < \delta < 0.6$.
- General note 2: If the user assigns a value $\delta \leq 0$, default values are considered.
- Note 1: By default, $\delta = 0.42$ for linear elements, $\delta = 0.23$ for quadratic elements, and $\delta = 0.14$ for cubic elements.

Table 2.6: List of options and values in section `bem formulation boundaries`

input.dat

```
...

[bem formulation over boundaries]
boundary 1: sbie
boundary 2: sbie
boundary 3: sbie
boundary 4: sbie
boundary 5: sbie
boundary 6: sbie_boundary_mca 0.1
```

2.3 Low-level entities of the model (mesh – geometric entities)

The low-level entities of the model are the elementary geometric entities (mesh) of the model: nodes, elements and parts. By default, the mesh is read from the input file by writing by hand the sections `[nodes]`, `[elements]` and `[parts]`, as explained below. However, there are other ways to create and read the mesh.

We have written a template file `*.bas` for the GiD pre- and post-processor, which allows GiD to produce a mesh file in our native mesh file format. Then, you can copy the contents of the generated file to the input file, or use `mesh_file_mode = 1 "filepath"` option in `[settings]` to indicate the format and path to the file. Each "layer" in GiD jargon corresponds to our concept of "part".

The program can read directly a mesh file from the Gmsh pre- and post-processor (MSH file format version 2.2), by using the `mesh_file_mode = 2 "filepath"` option in `[settings]` to indicate the format and path to the file. Each "physical" entity in their Gmsh jargon corresponds to our concept of "part".

2.3.1 Section `nodes`

In this section, all the nodes of the model are defined. Nodes are the most elementary part of the mesh, and they carry geometrical information (position of nodes), but also a functional/physical information (value of displacement, traction, etc. at that position).

The first line indicates the number of nodes. Then, one line per node indicating the identifier of the node and its coordinates. The general format of this section is:

general format of section [nodes]

```
[nodes]
<n = number of nodes>
<node 1 identifier> <x> <y> <z>
<node 2 identifier> <x> <y> <z>
...
<node n identifier> <x> <y> <z>
```

2.3.2 Section `elements`

In this section, all the elements of the model are defined. The elements are the fundamental part of the mesh, they allow the definition of an interpolation of the geometry and physical variables (displacements, tractions, etc.) supported on the nodes. On the other hand, high-level entities like boundaries and subdomains are built using a connected set of elements, which here it is called a "part".

The format of this section is very similar to the corresponding section of the Gmsh file format. The first line indicates the number of elements. Then, one line per element indicating:

- Element identifier.
- Type of element. It could be introduced via a string: `line2`, `line3`, `tri3`, `tri6`, `quad4`, `quad8`, `quad9`; or a number: `1`, `8`, `2`, `9`, `3`, `16`, `10`; respectively. Note that the numbers correspond to the numbers used by the `gmsh` file format.
- Number of auxiliary tags (greater than 0).
- List of tags, where the first auxiliary tag is mandatory, and corresponds to the identifier of the part which the element belongs. The rest of the tags are optional and they are read, but they are not used.
- A list of identifiers corresponding to the nodes of the element.

The general format of this section is:

general format of section [elements]

```
[elements]
<n = number of elements>
<element 1 identifier> <type> <num. of tags> <list of tags> <list of nodes identifiers>
<element 2 identifier> <type> <num. of tags> <list of tags> <list of nodes identifiers>
...
<element n identifier> <type> <num. of tags> <list of tags> <list of nodes identifiers>
```

Let's show an example of a mesh with 10 elements of the type `line3` (quadratic line element), where elements 1 to 5 belongs to part 1, and elements 6 to 10 belongs to part 2:

input.dat

```
...

[elements]
10
 3 line3 1 1  1  3  2
 2 line3 1 1  3  5  4
 1 line3 1 1  5  7  6
 4 line3 1 1  7  9  8
 5 line3 1 1  9 11 10
 6 line3 1 2 12 14 13
10 line3 1 2 14 16 15
```

```
 8 line3 1 2 16 18 17
 9 line3 1 2 18 20 19
 7 line3 1 2 20 22 21

...
```

Note that given that we use an identifier for each element, the order is unimportant. The same happens with other model entities such as nodes and parts.

2.3.3 Section `parts`

In this section, all the parts of the model are defined. A part is a connected set of elements. Hence, an element belongs only to one part, and one part can contain several elements. Note that the relationship between elements and parts was done when defining the elements. The general format of this section is:

general format of section [parts]

```
[parts]
<n = number of parts>
<part 1 identifier> <part name>
<part 2 identifier> <part name>
...
<part n identifier> <part name>
```

where `<part name>` is a string which allows labelling the part for easy identification. However, this is not used in the rest of the case input file.

2.4 High-level entities of the model (physical entities)

The high-level entities of the model are the classical topological entities: boundaries, and regions (or subdomains); which are used to assign material properties and conditions to different parts of the model. For convenience, four entities are defined: `boundaries` (boundary element boundaries), `be bodyloads` (body loads within a boundary element region), `fe subregions` (finite element subregions) and `regions`. These are defined by writing the corresponding sections `[boundaries]`, `[be bodyloads]`, `[fe subregions]` and `[regions]`, which are explained in the below.

2.4.1 Section `materials`

This section allows the definition of a set of materials and its properties. The type of materials available are three linear elastic materials: inviscid fluid, elastic solid, and Biot's poroelastic medium [**?**]. Each one of these represent a very different kind of material. The first one represents an acoustic medium, where only longitudinal P waves can propagate. It is therefore appropriate to model sound propagation through fluids like air or water. The second one is the common material for modelling structures, soils, etc. In a continuum region, longitudinal P and transverse S waves can propagate. The last one represent a porous medium where a compressible fluid is present inside a elastic frame. In a continuum region, two types of longitudinal waves (P1 and P2) and transverse S waves can propagate.

The section format is a sequence of numbers and strings, whose format can be described as follows:

general format of section [materials]

```
[materials]
<n = number of materials>
<material 1 identifier> <type of material> <property> <value> <property> <value> ...
<material 2 identifier> <type of material> <property> <value> <property> <value> ...
...
<material n identifier> <type of material> <property> <value> <property> <value> ...
```

The first line must contain the number of materials to be considered. Next, there must be as many lines as the number of materials defined. Each line starts with the material identifier (a integer greater than 0). Then, it follows with a string indicating the type of material to be defined in that line: `fluid`, `elastic_solid`, or `biot_poroelastic_medium`. After defining the type of material, if follows several pairs of string (property symbol) and real value (property value), which depends on the type of material.

If the type of material is `fluid` (which can be used only for time harmonic analysis), then it is necessary to define two between the three following properties: bulk modulus K, density ρ and wave propagation speed c. Optionally, you can define an articial hysteretic damping ratio ξ, which is zero by default. See Table 2.7 for more details.

Property	Math symbol	Plain text symbol
Bulk modulus	K	`K`
Density	ρ	`rho`
Wave propagation speed	$c = \sqrt{K/\rho}$	`c`
Artificial hysteretic damping ratio	ξ	`xi`

Table 2.7: Properties of an inviscid fluid (`fluid`). Only usable for time harmonic analysis. At least two between K, ρ and c must be defined. ξ is optional ($\xi = 0$ by default).

If the type of material is `elastic_solid`, then it is necessary to define two of the five following properties: Young's modulus E, bulk modulus K, Lamé's first parameter, λ, shear modulus μ and Poisson's ratio ν. If a time harmonic analysis is going to be performed, it is mandatory to define the density. Optionally, you can define the hysteretic damping ratio ξ, which is zero by default. See Table 2.8 for more details.

Property	Math symbol	Plain text symbol
Young's modulus (elastic modulus)	E	`E`
Bulk modulus	K	`K`
Lamé's first parameter	λ	`lambda`
Lamé's second parameter (shear modulus)	μ	`mu`
Poisson's ratio	ν	`nu`
Density	ρ	`rho`
Hysteretic damping ratio	ξ	`xi`

Table 2.8: Properties of an elastic solid (`elastic_solid`). At least two between E, K, λ, μ and ν must be defined. ρ is mandatory for time harmonic analysis. ξ is optional ($\xi = 0$ by default).

If the type of material is `biot_poroelastic_medium`, then it is necessary to define two of the five following properties of the solid phase: Young's modulus E, bulk modulus K, Lamé's first parameter, λ, shear modulus μ and Poisson's ratio ν. It is mandatory to define the solid and fluid phases densities, as well as the porosity, coupling parameters, additional density and dissipation constant. Optionally, you can define the solid phase hysteretic damping ratio ξ, which is zero by default. See Table 2.9 for more details.

Let's show a very simple example where we have two materials: water ($c = 1480$ m/s, $\rho = 1000$ kg/m^3) and soil modelled as an elastic solid ($E = 60$ MPa, $\nu = 0.4$, $\rho = 2000$ kg/m^3, $\xi = 5\%$). In such a case, the section should be written as:

input.dat

```
...

[materials]
2
1 fluid c 1480. rho 1000.
2 elastic_solid E 60.d6 nu 0.4 rho 2000. xi 0.05

...
```

2.4.2 Section `cross sections`

In order to define a structural finite element four pieces of information are required:

1. **Material properties**. It defines the constitutive law, density, and other intrinsic material properties. When needed by the structural element, these properties are taken from the **material** assigned to the **region** where the element belongs.

2. **Reference geometry**. It defines the mid-line for beams, mid-surface for plates/shells, etcetera. This information is provided by the defined **mesh**.

Property	Symbol	Plain text symbol
Solid phase		
Young's modulus (elastic modulus)	E	`E`
Bulk modulus	K	`K`
Lamé's first parameter	λ	`lambda`
Lamé's second parameter (shear modulus)	μ	`mu`
Poisson's ratio	ν	`nu`
Hysteretic damping ratio	ξ	`xi`
Solid density	ρ_s	`rhos`
Fluid phase and coupling parameters		
Fluid density	ρ_f	`rhof`
Porosity	ϕ	`phi`
Biot's coupling parameter	Q	`Q`
Biot's coupling parameter	R	`R`
Additional apparent density	ρ_a	`rhoa`
Dissipation constant	b	`b`

Table 2.9: Properties of a Biot's poroelastic medium (`biot_poroelastic_medium`). At least two between E, K, λ, μ and ν must be defined. ξ is optional ($\xi = 0$ by default). The other properties are mandatory.

3. **Structural element type**. It defines the physical behaviour of the element. A given line element could be a simple bar element with only axial tension/compression stiffness, or a spring-dashpot or a beam.

4. **Cross section properties**. It complements the information provided by the mesh, allowing a full description of the structural element. The cross section properties could be given in a detailed manner (cross section dimensions), or by providing effective geometrical properties (area, inertias, ...), or by providing effective mechanical properties (stiffnesses, masses, ...).

In this section, both the structural element type and the cross section properties are defined. Both aspects are globally denoted as a "cross section".

The first line of this data section defines the number of different cross sections to be defined. Next, there must be as many lines as the number of cross sections defined. Each of these lines start with the type of structural element to be defined, then the number of FE subregions where this type of structural element is assigned, followed by the list of FE subregions. Next, depending on the type of structural element, different cross section properties are defined. Therefore, the general format of this section is:

general format of section [cross sections]

```
[cross sections]
<# cross sections>
<type of structural element 1> <# FE subregions> <list of FE subregions ids.> <cross section properties>
<type of structural element 2> <# FE subregions> <list of FE subregions ids.> <cross section properties>
...
```

In the following subsections, the previous data format will be condensed to:

general format of section [cross sections]

```
[cross sections]
<# cross sections>
<type of structural element 1> <FE subregions> <cross section properties>
<type of structural element 2> <FE subregions> <cross section properties>
...
```

where `<FE subregions>` in reality denotes `<# FE subregions> <list of FE subregions ids.>`.

The list of available types of structural elements are:

- Discrete element:
  - Discrete axial spring-dashpot (`spring-dashpot`).

  - General discrete translational spring-dashpot (`distra`).
  - General discrete rotational/translational spring-dashpot (`disrotra`).
  - Discrete point mass (`pmass`).

- Bar finite element (`bar`).

- Beam finite element:

  - Straight beam finite element (`strbeam`).
  - Curved beam finite element based on the degeneration from solid (`degbeam`).

- Shell finite element based on the degeneration from solid (`degshell`).

When considering a 3D analysis, most of the structural elements require orienting their cross section, which leads to a particular cartesian local basis (local axes $x'y'z'$) where the relevant dimensions and the internal stress resultants are defined. In the case of one-dimensional structural elements, such as discrete general springs-dashpots or beams, the orientation of the y' axis is defined with the help of an auxiliary reference vector $\mathbf{y}'_{\text{ref}}$, see Figure 2.1, where the final y' axis is located on the plane defined by $\mathbf{y}'_{\text{ref}}$ and the x' axis. Note that the x' axis is defined by the tangent direction of the element mid-line. In the case of two-dimensional structural elements, i.e. plates and shells, the procedure is similar, see Figure 2.2, but in this case the x' axis is defined by using an auxiliary reference vector $\mathbf{x}'_{\text{ref}}$, being the final x' axis on the tangent plane defined by the z' axis (coincident with the normal vector at the point) and the plane defined by $\mathbf{x}'_{\text{ref}}$ and the z' axis.

2.4.2.1 Discrete axial spring-dashpot (`spring-dashpot`)

This type of structural finite element is a conventional axial spring-dashpot element between two nodes, and it is denoted as `spring-dashpot` in the data section. It is a particular case of `distra` element type, and much more common. The data introduction is also simpler. Taking into account that `<# FE subregions> <list of FE subregions ids.>` is here denoted as `<FE subregions>`, the syntax to describe this type of elements is:

- In static analysis, $k_{x'}$ is the spring stiffness:

  format of section [cross sections] for spring-dashpot elements
  ```
  [cross sections]
  ...
  spring-dashpot <FE subregions> <kx'>
  ...
  ```

- In time harmonic analysis, the stiffness $k_{x'}$ must be introduced as a complex number to take into account a possible hysteretic damping, and $c_{x'}$ is the viscous damping coefficient:

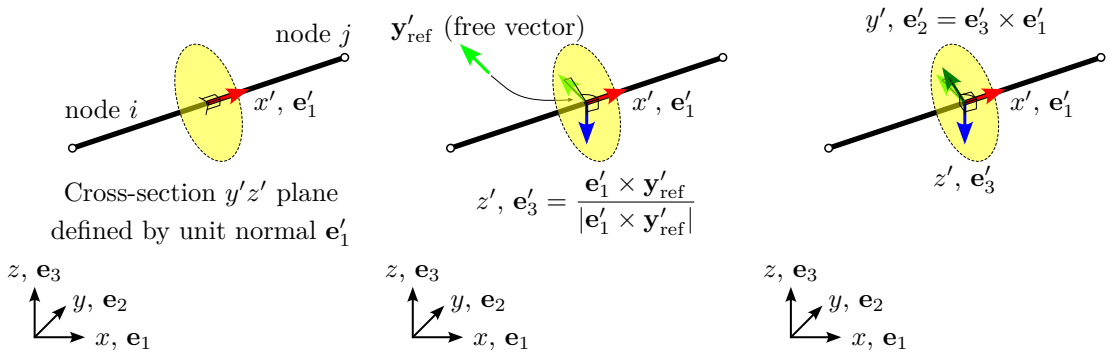
  format of section [cross sections] for spring-dashpot elements
  ```
  [cross sections]
  ...
  spring-dashpot <FE subregions> <kx'> <cx'>
  ...
  ```

2.4.2.2 General discrete translational spring/dashpot (`distra`)

This type of structural finite element introduces a generalized translational spring-dashpot between two nodes, and it is denoted as `distra` in the data section. It requires defining a 2-node line element. Each element node has 2 or 3 translational degrees of freedom respectively for two- and three-dimensional analyses, see Figure 2.3. For static analysis, stiffnesses in global or local axes are required. For time harmonic analysis, it is required to use complex stiffnesses to introduce stiffnesses (real part) and hysteretic damping (imaginary part), and also viscous damping coefficients.

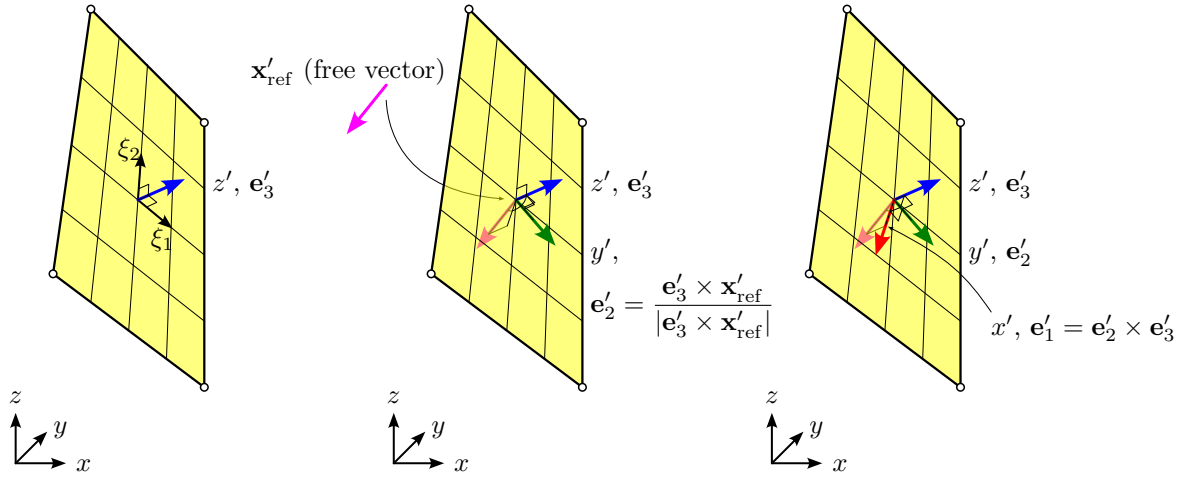
Mechanical properties can be defined in local or global axes:

- **Local axes**. It allows to establish the stiffnesses and viscous damping coefficients in local axes, i.e. axial spring-dashpot in x' direction ($k_{x'}$, $c_{x'}$) and lateral spring-dashpots in y' and z' directions



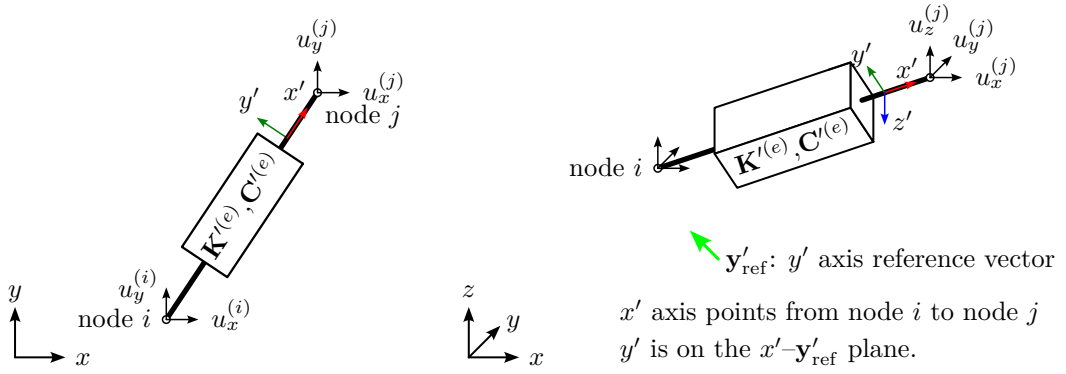
(a) $\mathbf{e}'_1$ from element (b) $\mathbf{e}'_3$ from $\mathbf{e}'_1 \times \mathbf{y}'_{\text{ref}}$ (c) $\mathbf{e}'_2$ from $\mathbf{e}'_3 \times \mathbf{e}'_1$

Figure 2.1: Steps for constructing the local cartesian basis $x'y'z'$ of one-dimensional structural elements from a y' axis reference vector $\mathbf{y}'_{\text{ref}}$



(a) $\mathbf{e}'_3$ from element (b) $\mathbf{e}'_2$ from $\mathbf{e}'_3 \times \mathbf{x}'_{\text{ref}}$ (c) $\mathbf{e}'_1$ from $\mathbf{e}'_2 \times \mathbf{e}'_3$

Figure 2.2: Steps for constructing the local cartesian basis $x'y'z'$ of two-dimensional structural elements from a x' axis reference vector $\mathbf{x}'_{\text{ref}}$



(a) 2D analysis (b) 3D analysis

Figure 2.3: Degrees of freedom and local axis definition of discrete translational springs/dashpots (**distra**)

($k_{y'}$, $c_{y'}$, $k_{z'}$, $c_{z'}$). In two-dimensional analysis, the local stiffness and damping coefficient matrices are:

$$\mathbf{K}' = \begin{bmatrix} k_{x'} & 0 & -k_{x'} & 0 \\ 0 & k_{y'} & 0 & -k_{y'} \\ -k_{x'} & 0 & k_{x'} & 0 \\ 0 & -k_{y'} & 0 & k_{y'} \end{bmatrix}, \quad \mathbf{C}' = \begin{bmatrix} c_{x'} & 0 & -c_{x'} & 0 \\ 0 & c_{y'} & 0 & -c_{y'} \\ -c_{x'} & 0 & c_{x'} & 0 \\ 0 & -c_{y'} & 0 & c_{y'} \end{bmatrix} \tag{2.6}$$

In three-dimensional analysis:

$$\mathbf{K}' = \begin{bmatrix} k_{x'} & 0 & 0 & -k_{x'} & 0 & 0 \\ 0 & k_{y'} & 0 & 0 & -k_{y'} & 0 \\ 0 & 0 & k_{z'} & 0 & 0 & -k_{z'} \\ -k_{x'} & 0 & 0 & k_{x'} & 0 & 0 \\ 0 & -k_{y'} & 0 & 0 & k_{y'} & 0 \\ 0 & 0 & -k_{z'} & 0 & 0 & k_{z'} \end{bmatrix}, \quad \mathbf{C}' = \begin{bmatrix} c_{x'} & 0 & 0 & -c_{x'} & 0 & 0 \\ 0 & c_{y'} & 0 & 0 & -c_{y'} & 0 \\ 0 & 0 & c_{z'} & 0 & 0 & -c_{z'} \\ -c_{x'} & 0 & 0 & c_{x'} & 0 & 0 \\ 0 & -c_{y'} & 0 & 0 & c_{y'} & 0 \\ 0 & 0 & -c_{z'} & 0 & 0 & c_{z'} \end{bmatrix} \tag{2.7}$$

In this latter case, in order to uniquely define the local axis, a reference vector $\mathbf{y}'_{\text{ref}}$ is required for establishing the local y' axis, see Figures 2.2 and 2.3 for more details.

- **Global axes**. It allows to establish the stiffnesses and viscous damping coeficients directly in global axes, i.e. springs and dashpots in x, y and z directions: k_x, c_x, k_y, c_y, k_z and, c_z. Therefore, the relative position of element nodes are unimportant.

Taking into account that `<# FE subregions> <list of FE subregions ids.>` is here denoted as `<FE subregions>`, the syntax to describe this type of elements is:

- In a 2D static analysis, this type of elements can be introduced in local or in global axes as follows:

format of section [cross sections] for distra elements

```
[cross sections]
...
distra <FE subregions> local <kx'> <ky'>
distra <FE subregions> global <kx> <ky>
...
```

- In a 3D static analysis, this type of elements can be introduced in local (requires $\mathbf{y}'_{\text{ref}}$) or in global axes as follows:

format of section [cross sections] for distra elements

```
[cross sections]
...
```

```
distra <FE subregions> local <kx'> <ky'> <kz'> <y'refx> <y'refy> <y'refz>
distra <FE subregions> global <kx> <ky> <kz>
...
```

- In a 2D time harmonic analysis, this type of elements can be introduced in local or in global axes as follows (note that $k_{x'}$ and $k_{y'}$ must be complex numbers):

format of section [cross sections] for distra elements

```
[cross sections]
...
distra <FE subregions> local <kx'> <ky'> <cx'> <cy'>
distra <FE subregions> global <kx> <ky> <cx> <cy>
...
```

- In a 3D time harmonic analysis, this type of elements elements can be introduced in local (requires $\mathbf{y}'_{\text{ref}}$) or in global axes as follows (note that $k_{x'}$, $k_{y'}$ and $k_{z'}$ must be complex numbers):

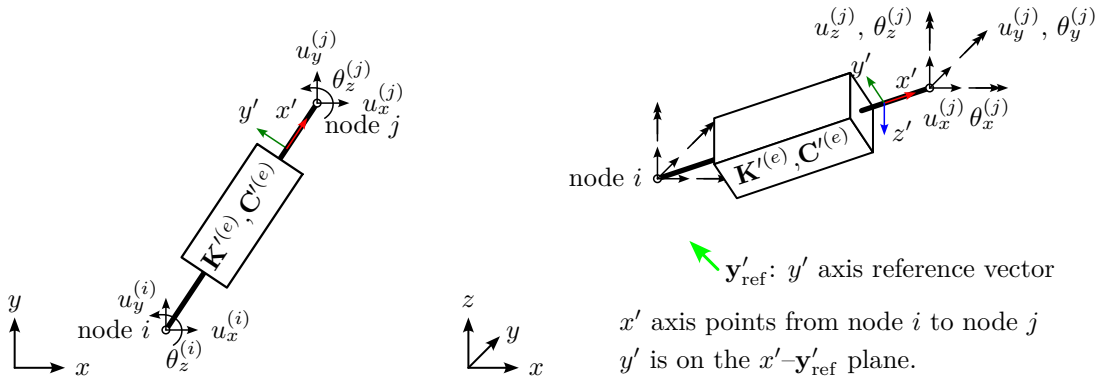
format of section [cross sections] for distra elements

```
[cross sections]
...
distra <FE subregions> local <kx'> <ky'> <kz'> <cx'> <cy'> <cz'> <y'refx> <y'refy> <y'refz>
distra <FE subregions> global <kx> <ky> <kz> <cx> <cy> <cz>

...
```

2.4.2.3 General discrete rotational/translational spring-dashpot (`disrotra`)

This type of structural finite element introduces a generalized translational/rotational spring-dashpot between two nodes, and it is denoted as `disrotra` in the data section. It requires defining a 2-node line element. Each element node has 3 or 6 degrees of freedom respectively for two- and three-dimensional analyses, see Figure 2.4. For static analysis, stiffnesses in global or local axes are required. For time harmonic analysis, it is required to use complex stiffnesses to introduce stiffnesses (real part) and hysteretic damping (imaginary part), and also viscous damping coefficients. This type of element allows modelling a customized beam, a simple spring-dashpo model of the soil-foundation interaction, etcetera.



(a) 2D analysis (b) 3D analysis

Figure 2.4: Degrees of freedom and local axis definition of discrete translational/rotational springs/dashpots (**disrotra**)

Mechanical properties can be defined in local or global axes:

- **Local axes**. It allows to establish the stiffnesses and viscous damping coefficients in local axes:
  - Axial (x' translation): $k_{x'}$, $c_{x'}$.
  - Torsional (x' rotation): $k_{rx'}$, $c_{rx'}$.
  - Bending on $x'y'$ plane (y' translation – z' rotation): $k_{y'}$, $c_{y'}$, $k_{rz'}$, $c_{rz'}$, $k_{y'rz'}$, $c_{y'rz'}$.
  - Bending on $x'z'$ plane (z' translation – y' rotation): $k_{z'}$, $c_{z'}$, $k_{ry'}$, $c_{ry'}$, $k_{z'ry'}$, $c_{z'ry'}$

In two-dimensional analysis, the local stiffness and damping coefficient matrices are:

$$\mathbf{K}' = \left[\begin{array}{ccc|ccc} k_{x'} & 0 & 0 & -k_{x'} & 0 & 0 \\ 0 & k_{y'} & k_{y'rz'} & 0 & -k_{y'} & k_{y'rz'} \\ 0 & k_{y'rz} & k_{rz'} & 0 & -k_{y'rz} & 2\frac{k_{y'rz'}^2}{k_{y'}} - k_{rz'} \\ \hline -k_{x'} & 0 & 0 & k_{x'} & 0 & 0 \\ 0 & -k_{y'} & -k_{y'rz'} & 0 & k_{y'} & -k_{y'rz'} \\ 0 & k_{y'rz'} & 2\frac{k_{y'rz'}^2}{k_{y'}} - k_{rz'} & 0 & -k_{y'rz'} & k_{rz'} \end{array}\right] \tag{2.8}$$

$$\mathbf{C}' = \left[\begin{array}{ccc|ccc} c_{x'} & 0 & 0 & -c_{x'} & 0 & 0 \\ 0 & c_{y'} & c_{y'rz'} & 0 & -c_{y'} & c_{y'rz'} \\ 0 & c_{y'rz} & c_{rz'} & 0 & -c_{y'rz} & 2\frac{c_{y'rz'}^2}{c_{y'}} - c_{rz'} \\ \hline -c_{x'} & 0 & 0 & c_{x'} & 0 & 0 \\ 0 & -c_{y'} & -c_{y'rz'} & 0 & c_{y'} & -c_{y'rz'} \\ 0 & c_{y'rz'} & 2\frac{c_{y'rz'}^2}{c_{y'}} - c_{rz'} & 0 & -c_{y'rz'} & c_{rz'} \end{array}\right] \tag{2.9}$$

In three-dimensional analysis:

$$\mathbf{K}' = \left[\begin{array}{cccccc|cccccc} k_{x'} & 0 & 0 & 0 & 0 & 0 & -k_{x'} & 0 & 0 & 0 & 0 & 0 \\ & k_{y'} & 0 & 0 & 0 & k_{y'rz'} & 0 & -k_{y'} & 0 & 0 & 0 & k_{y'rz'} \\ & & k_{z'} & 0 & k_{z'ry'} & 0 & 0 & 0 & -k_{z'} & 0 & -k_{z'ry'} & 0 \\ & & & k_{rx'} & 0 & 0 & 0 & 0 & 0 & -k_{rx'} & 0 & 0 \\ & \text{sym} & & & k_{ry'} & 0 & 0 & 0 & k_{z'ry'} & 0 & 2\frac{k_{z'ry'}^2}{k_{z'}} - k_{ry'} & 0 \\ & & & & & k_{rz'} & 0 & -k_{y'rz'} & 0 & 0 & 0 & 2\frac{k_{y'rz'}^2}{k_{y'}} - k_{rz'} \\ \hline & & & & & & k_{x'} & 0 & 0 & 0 & 0 & 0 \\ & & & & & & & k_{y'} & 0 & 0 & 0 & k_{y'rz'} \\ & & & & & & & & k_{z'} & 0 & k_{z'ry'} & 0 \\ & & & & & & & & & k_{rx'} & 0 & 0 \\ & \text{sym} & & & & & & \text{sym} & & & k_{ry'} & 0 \\ & & & & & & & & & & & k_{rz'} \end{array}\right] \tag{2.10}$$

An similarly for $\mathbf{C}'$.

In order to uniquely define the local axis, for three-dimensional analysis a reference vector $\mathbf{y}'_{\text{ref}}$ is required for establishing the local y' axis, see Figures 2.2 and 2.3 for more details.

- **Global axes**. It allows to establish the stiffnesses and viscous damping coeficients directly in global axes, and thus the relative position of element nodes are unimportant.

Taking into account that `<# FE subregions> <list of FE subregions ids.>` is here denoted as `<FE subregions>`, the syntax to describe this type of elements is:

- In a 2D static analysis, this type of elements can be introduced in local or in global axes as follows:

format of section [cross sections] for disrotra elements

```
[cross sections]
...
disrotra <FE subregions> local <kx'> <ky'> <krz'> <ky'rz'>
disrotra <FE subregions> global <kx> <ky> <krz> <kyrz>
...
```

- In a 3D static analysis, this type of elements can be introduced in local (requires $\mathbf{y}'_{\text{ref}}$) or in global axes as follows:

format of section [cross sections] for disrotra elements

```
[cross sections]
...
disrotra <FE subregions> local <kx'> <ky'> <kz'> <krx'> <kry'> <krz'> <ky'rz'> <kz'ry'>
(continuation of previous line) <y'refx> <y'refy> <y'refz>
disrotra <FE subregions> global <kx> <ky> <kz> <krx> <kry> <krz> <kyrz> <kzry>
...
```

- In a 2D time harmonic analysis, this type of elements can be introduced in local or in global axes as follows (note that all stiffnesses $k_\square$ must be complex numbers):

format of section [cross sections] for disrotra elements

```
[cross sections]
...
disrotra <FE subregions> local <kx'> <ky'> <krz'> <ky'rz'> <cx'> <cy'> <crz'> <cy'rz'>
disrotra <FE subregions> global <kx> <ky> <krz> <kyrz> <cx> <cy> <crz> <cyrz>
...
```

- In a 3D time harmonic analysis, this type of elements elements can be introduced in local (requires $\mathbf{x}'_{\text{ref}}$) or in global axes as follows (note that all stiffnesses $k_\square$ must be complex numbers):

format of section [cross sections] for disrotra elements

```
[cross sections]
...
disrotra <FE subregions> local <kx'> <ky'> <kz'> <krx'> <kry'> <krz'> <ky'rz'> <kz'ry'>
(continuation of previous line) <cx'> <cy'> <cz'> <crx'> <cry'> <crz'> <cy'rz'> <cz'ry'>
(continuation of previous line) <y'refx> <y'refy> <y'refz>
disrotra <FE subregions> global <kx> <ky> <kz> <krx> <kry> <krz> <kyrz> <kzry>
(continuation of previous line) <cx> <cy> <cz> <crx> <cry> <crz> <cyrz> <czry>
...
```

2.4.2.4 Discrete point mass (pmass)

A point mass element can be used for simplifying the interaction between a given body and a supporting structure to the body inertial forces reduced to the contact point. Therefore, the reduced body is considered rigid. It is important to note that it is used only in time harmonic analysis, as it represents the inertial forces and not the body weight, which must be introduced as a load.

This type of structural element requires defining a 0-dimensional element in the mesh, i.e. 1-node element. For three-dimensional analysis, it is introduced in the model by adding the corresponding inertial forces of the body e to a given node i as:

$$\mathbf{F}^{(e)}_{\text{inertial}} = -\mathbf{M}^{(e)} \cdot \ddot{\mathbf{a}}^{(i)} = \begin{bmatrix} \mathbf{M}_0^{(e)} & \mathbf{M}_1^{(e)} \\ \left(\mathbf{M}_1^{(e)}\right)^T & \mathbf{M}_2^{(e)} \end{bmatrix} \cdot \begin{bmatrix} u_x^{(i)} & u_y^{(i)} & u_z^{(i)} & \theta_x^{(i)} & \theta_y^{(i)} & \theta_z^{(i)} \end{bmatrix}^T \tag{2.11}$$

where $\mathbf{M}^{(e)}$ is the generalized mass matrix, which can be divided into three matrices:

$$\mathbf{M}_0^{(e)} = \mathbf{I} \int_\Omega \rho \, \mathrm{d}\Omega = \begin{bmatrix} M^{(e)} & 0 & 0 \\ 0 & M^{(e)} & 0 \\ 0 & 0 & M^{(e)} \end{bmatrix} \tag{2.12}$$

$$\mathbf{M}_1^{(e)} = \int_\Omega \rho \begin{bmatrix} 0 & r_3 & -r_2 \\ -r_3 & 0 & r_1 \\ r_2 & -r_1 & 0 \end{bmatrix} \mathrm{d}\Omega = \begin{bmatrix} 0 & B_{xy}^{(e)} & -B_{xz}^{(e)} \\ -B_{xy}^{(e)} & 0 & B_{yz}^{(e)} \\ B_{xz}^{(e)} & -B_{yz}^{(e)} & 0 \end{bmatrix} \tag{2.13}$$

$$\mathbf{M}_2^{(e)} = \int_\Omega \rho \begin{bmatrix} r_2^2 + r_3^2 & -r_1 r_2 & -r_1 r_3 \\ -r_2 r_1 & r_1^2 + r_3^2 & -r_2 r_3 \\ -r_3 r_1 & -r_3 r_2 & r_1^2 + r_2^2 \end{bmatrix} \mathrm{d}\Omega = \begin{bmatrix} J_{xx}^{(e)} & -J_{xy}^{(e)} & -J_{xz}^{(e)} \\ -J_{xy}^{(e)} & J_{yy}^{(e)} & -J_{yz}^{(e)} \\ -J_{xz}^{(e)} & -J_{yz}^{(e)} & J_{zz}^{(e)} \end{bmatrix} \tag{2.14}$$

where Ω is the domain of the body e, ρ is its the density distribution, and $r_j = x_j - x_j^{(i)}$ is the position vector of each body point with respect to the reduction point/node i. $\mathbf{M}_0$ is the translational inertia

(zero moment of mass), $\mathbf{M}_1$ represents the inertia forces and moments due to the mass imbalance (first moments of mass), and $\mathbf{M}_2$ represents the rotational inertia (second moments of mass). If the node i coincides with the center of mass of the body, then $\mathbf{M}_1^{(e)}$ is a zero matrix. If additionally the body has three symmetry planes or if its principal axes coincides with the global axes, then $\mathbf{M}_2^{(e)}$ is a diagonal matrix. For time harmonic analyses with time factor $e^{i\omega t}$, the generalized displacements acceleration becomes:

$$\mathbf{F}_{\text{inertial}}^{(e)} = -\mathbf{M}^{(e)} \cdot \ddot{\mathbf{a}}^{(i)} = \omega^2 \mathbf{M}^{(e)} \cdot \mathbf{a}^{(i)} \tag{2.15}$$

In the data input, each mass matrix term M_{jk} have to be defined. In 2D analysis, the mass matrix is a 3×3 matrix because of the degrees of freedom involved:

$$\mathbf{F}_{\text{inertial}}^{(e)} = \omega^2 \mathbf{M}^{(e)} \cdot \mathbf{a}^{(i)} = \omega^2 \begin{bmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \\ M_{31} & M_{32} & M_{33} \end{bmatrix} \cdot \begin{bmatrix} u_x^{(i)} \\ u_y^{(i)} \\ \theta_z^{(i)} \end{bmatrix} \tag{2.16}$$

In 3D analysis, the mass matrix is a 6×6 matrix:

$$\mathbf{F}_{\text{inertial}}^{(e)} = \omega^2 \mathbf{M}^{(e)} \cdot \mathbf{a}^{(i)} = \omega^2 \begin{bmatrix} M_{11} & M_{12} & M_{13} & M_{14} & M_{15} & M_{16} \\ M_{21} & M_{22} & M_{23} & M_{24} & M_{25} & M_{26} \\ M_{31} & M_{32} & M_{33} & M_{34} & M_{35} & M_{36} \\ M_{41} & M_{42} & M_{43} & M_{44} & M_{45} & M_{46} \\ M_{51} & M_{52} & M_{53} & M_{54} & M_{55} & M_{56} \\ M_{61} & M_{62} & M_{63} & M_{64} & M_{65} & M_{66} \end{bmatrix} \cdot \begin{bmatrix} u_x^{(i)} \\ u_y^{(i)} \\ u_z^{(i)} \\ \theta_x^{(i)} \\ \theta_y^{(i)} \\ \theta_z^{(i)} \end{bmatrix} \tag{2.17}$$

In order to simplify the data introduction, when a balanced condition can be assumed only the M_{kk} non-zero entries have to be defined. Otherwise, a general nonbalanced condition requires defining all matrix entries. Therefore, the data introduction can be summarized as follows:

- In a **2D analysis**, you can introduce only the 3 non-zero diagonal terms for a balanced mass condition, or the full 3×3 matrix for a general unbalanced condition:

format of section [cross sections] for pmass elements

```
[cross sections]
...
pmass <FE subregions> unbalanced <M11> <M21> <M31> <M21> <M22> <M23> <M13> <M23> <M33>
pmass <FE subregions> balanced <M11> <M22> <M33>
...
```

- In a **3D analysis**, you can introduce only the 6 non-zero diagonal terms for a balanced mass condition, or the full 6×6 matrix for a general unbalanced condition:

format of section [cross sections] for pmass elements

```
[cross sections]
...
pmass <FE subregions> unbalanced <M11> <M21> <M31> <M41> <M51> <M61> <M12> <M22> ... <M66>
pmass <FE subregions> balanced <M11> <M22> <M33> <M44> <M55> <M66>
...
```

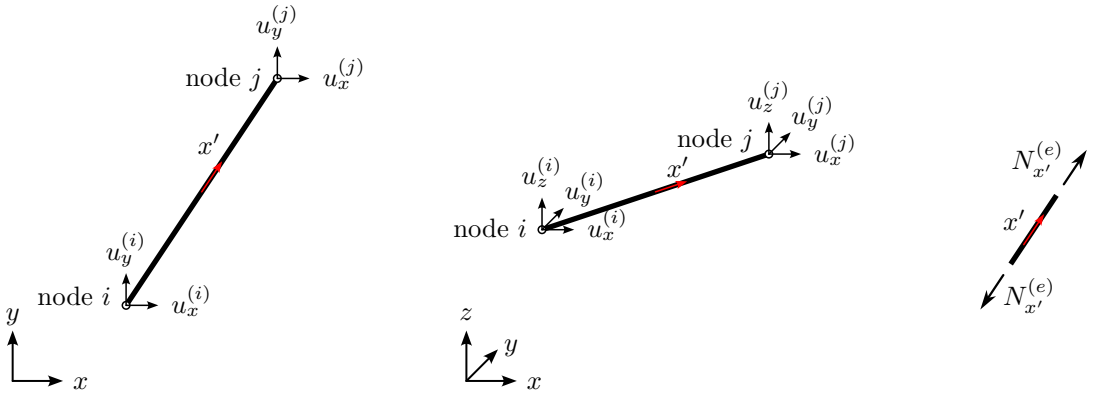
2.4.2.5 Bar finite element (bar)

This type of structural finite element is a simple tension/compression member [**?**] between two nodes, and it is denoted as `bar` in the data section. It requires using a 2-node line element in the mesh. Each element node has 2 or 3 translational degrees of freedom respectively for two- and three-dimensional analyses, and the only stress resultant is the axial force $N_{x'}^{(e)}$, see Figure 2.5. For time harmonic analysis, a consistent mass matrix is used, and its density is taken from the **material** defined for the corresponding **fe subregion** and **region**.

The only required cross section parameter is the area, and it can be introduced in several ways:

format of section [cross sections] for bar elements

```
[cross sections]
...
bar <FE subregions> generic <area>
bar <FE subregions> circle <diameter>
bar <FE subregions> hollow_circle <outer diameter> <inner diameter>
bar <FE subregions> rectangle <width> <height>
...
```



(a) 2D analysis (b) 3D analysis (c) Axial force

Figure 2.5: Bar finite element description (`bar`)

2.4.2.6 Straight beam finite element (`strbeam`)

This type of structural finite element is a conventional straight beam finite element, and it is denoted as `strbeam` in the data section. By default, Timoshenko kinematic assumption (plane cross section after deformation) is assumed for bending, but the simpler Euler-Bernoulli theory is also available (cross section remains plane and perpendicular to the mid-line after deformation). It can be chosen by using as the type of structural element:

- Timoshenko beam theory: `strbeam` or `strbeam_t`
- Euler-Bernoulli beam theory: `strbeam_eb`.

The formulation is based on locking-free Timoshenko beam finite element of Friedman & Kostmatka [**?**], and Euler-Bernoulli assumptions are simply obtained by nullifying the transverse shear strain factor ($\phi = 0$). The element is built by superposition of axial, torsional and bending behaviour. Line elements of 2 or 3 nodes can be used in the mesh. For time harmonic analysis, a consistent mass matrix including rotational inertia is used, and its density is taken from the **material** defined for the corresponding **fe subregion** and **region**.

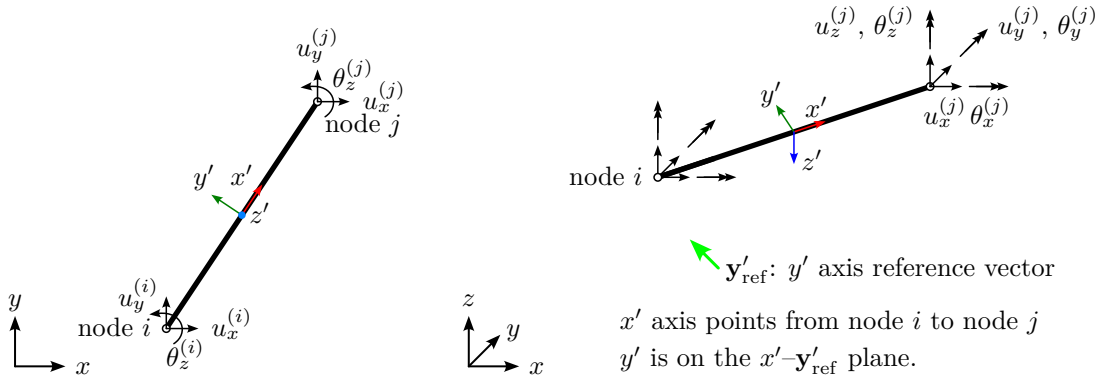
A symmetric cross section is assumed, and the required cross section parameters are:

- Area: A.
- Area moments of inertia: $I_{x'}$, $I_{y'}$, $I_{z'}$.
- Shear correction factors: $\kappa_{x'}$, $\kappa_{y'}$, $\kappa_{z'}$. They are used to calculate the effective shear stiffnesses by correcting the cross section parameters: $\kappa_{x'} I_{x'}$ (torsional constant), $\kappa_{y'} A$ (y' transverse shear area), $\kappa_{z'} A$ (z' transverse shear area).

In two-dimensional analysis, each node has 3 DOFs (u_x, u_y, θ_z), whereas in three-dimensional analysis each node has 6 DOFs (u_x, u_y, u_z, θ_x, θ_y, and θ_z), see Figure 2.6.

The data introduction can be summarized as follows:

- In a **2D analysis** under plane strain, a beam can be used to model an infinitely long ($h_{z'} = 1$) wall/plate/shell of thickness $h_{y'}$, or an equivalent structural system. It is required the cross section area A (usual application requires $A = h_{y'} \cdot 1$), the inertia $I_{z'}$ (usual application requires $I_{z'} = 1 \cdot h_{y'}^3/12$), and the shear correction factor $\kappa_{y'}$ (usual application requires $\kappa_{y'} = 5/6$):



(a) 2D analysis (b) 3D analysis

Figure 2.6: Degrees of freedom and local axis definition of straight beam finite elements (**strbeam**). Element with 2 node shown.

format of section [cross sections] for strbeam elements

```
[cross sections]
...
strbeam <FE subregions> <A> <I_z'> <kappa_y'>
...
```

- In a **3D analysis**, you can introduce a generic cross section, or a pre-defined cross section shape via its dimensions. The reference vector $\mathbf{y}'_{\text{ref}}$ for defining the y' axis is required (see Figure 2.1 for more details). Therefore, it can be introduced as follows:

format of section [cross sections] for strbeam elements

```
[cross sections]
...
strbeam <FE subregions> generic <A> <Ix'> <Iy'> <Iz'> <kax'> <kay'> <kaz'> <y'refx> <y'refy> <y'refz>
strbeam <FE subregions> circle <diameter> <y'refx> <y'refy> <y'refz>
strbeam <FE subregions> hollow_circle <outer diam> <inner diam> <y'refx> <y'refy> <y'refz>
strbeam <FE subregions> rectangle <width y'> <height z'> <y'refx> <y'refy> <y'refz>
...
```

For stress resultants definition, see Figure 2.8 and section about `degbeam`.

2.4.2.7 Beam finite element obtained from the degeneration of the solid (`degbeam`)

This type of structural finite element is a beam finite element obtained from the degeneration of the solid via Timoshenko kinematic assumption (plane cross section after deformation), and it is denoted as `degbeam` in the data section. A rectangular cross section of $h_{y'} \times h_{z'}$ is assumed. It requires using a 2, 3 or 4 node line element in the mesh. In order to avoid locking, a selective integration is used by default, but it can be changed in `element options` data section. For time harmonic analysis, a consistent mass matrix is used, and its density is taken from the **material** defined for the corresponding **fe subregion** and **region**.

Similarly to `strbeam`, in two-dimensional analysis each node has 3 DOFs (u_x, u_y, θ_z), and in three-dimensional analysis each node has 6 DOFs (u_x, u_y, u_z, θ_x, θ_y, and θ_z), see Figure 2.7.

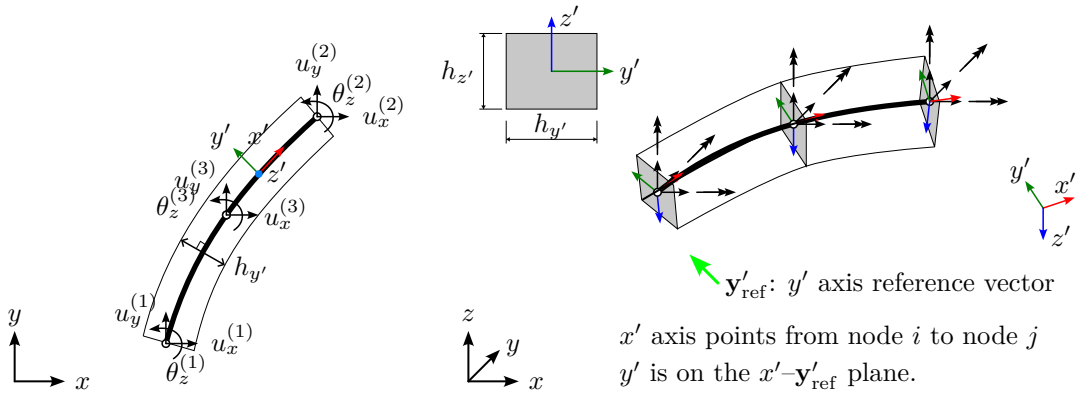
The data introduction can be summarized as follows:

- In a **2D analysis**, a plain strain condition is assumed (width is unitary), being the x' and y' axes contained in the plane and only the cross section height $h_{y'}$ is required:

format of section [cross sections] for degbeam elements

```
[cross sections]
...
degbeam <FE subregions> <hy'>
...
```

- In a **3D analysis**, the cross section dimensions $h_{y'}$ and $h_{z'}$ are required together with the reference vector $\mathbf{y}'_{\text{ref}}$ for the y' axis (see Figure 2.1 for more details):



(a) 2D analysis (b) 3D analysis

Figure 2.7: Degrees of freedom and local axis definition of beam finite elements degenerated from solid (**degbeam**). Element with 3 node shown.

format of section [cross sections] for degbeam elements

```
[cross sections]
...
degbeam <FE subregions> <hy'> <hz'> <y'ref x> <y'ref y> <y'ref z>
...
```

The stress resultants are defined as follows:

$$\begin{Bmatrix} N_{x'} \\ V_{y'} \\ V_{z'} \\ T_{x'} \\ M_{y'} \\ M_{z'} \end{Bmatrix} = \int_A \begin{Bmatrix} \sigma_{x'x'} \\ \tau_{x'y'} \\ \tau_{x'z'} \\ -z' \cdot \tau_{x'y'} + y' \cdot \tau_{x'z'} \\ z' \cdot \sigma_{x'x'} \\ -y' \cdot \tau_{x'x'} \end{Bmatrix} \mathrm{d}A \tag{2.18}$$

and they are shown in Figure 2.8.

2.4.2.8 Shell finite element obtained from the degeneration of the solid (`degshell`)

This type of structural finite element is a shell finite element obtained from the degeneration of the solid via Reissner-Mindlin kinematic assumption [**?**], and it is denoted as `degshell` in the data section. It requires using a triangular element (3 or 6 nodes) or a quadrilateral element (4, 8 or 9 nodes) in the mesh. By default, the locking-free Mixed Interpolation of Tensorial Components (MITC) family of elements of Bathe and co-workers [**?**, **?**, **?**, **?**, **?**, **?**] with full integration is used: MITC3i, MITC4, MITC6a, MITC8*, MITC9. Standard shell finite element (without MITC) can still be used by denoting them as `degshell_std`.

Each element node i has 3 translational DOF ($u_x^{(i)}$, $u_y^{(i)}$, $u_z^{(i)}$), and 2 local rotational DOFs ($\alpha^{(i)}$ and $\beta^{(i)}$ local rotations related to shell bending) or 3 global rotational DOFs ($\theta_x^{(i)}$, $\theta_y^{(i)}$ and $\theta_z^{(i)}$ global rotations). This shell model does not have rotational stiffness in the normal direction, the so called "drilling" stiffness, thus an artificial stiffness is added when using 6 DOF [**?**] in order to avoid any singularity. There are three situations where 6 DOFs are mandatory:

1. When imposing certain boundary conditions (for example symmetry conditions).
2. Along edges or joints between two shells with incompatible orientations.
3. Along joints between more than two shells, at beam-shell joints, and similar singular joints.

Therefore, in practical problems most element nodes can be 5 DOFs. Although we could have implemented only 6 DOFs element nodes (more homogeneous treatment), we have decided to reduce as much as possible the number of DOF since we do not use a sparse solver (it is not appropriate for boundary element matrices

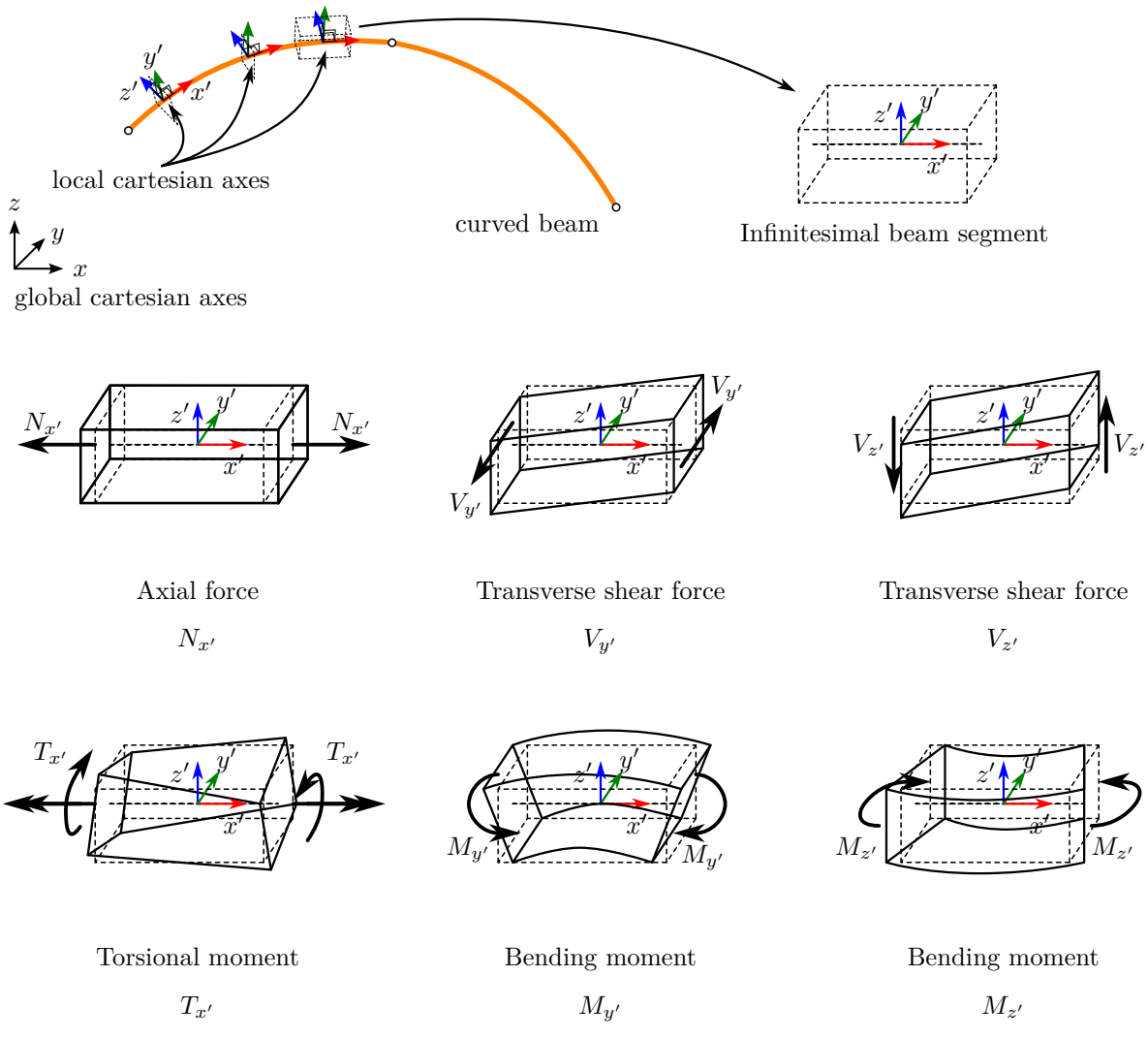


Figure 2.8: Beam stress resultants: $N_{x'}$, $V_{y'}$, $V_{z'}$, $T_{x'}$, $M_{y'}$, $M_{z'}$

which are dense). Internally, an automatic procedure for selecting 5 DOFs or 6 DOFs is implemented based on the conditions stated above. For time harmonic analysis, a consistent mass matrix is used, and its density is taken from the **material** defined for the corresponding **fe subregion** and **region**.

When defining `degshell` FE subregions, the data format of the cross section definition only requires the shell thickness and the $\mathbf{x}'_{\text{ref}}$ reference vector for establishing the x' local axis (see Figure 2.2 for more details):

format of section [cross sections] for degshell elements

```
[cross sections]
...
degshell <FE subregions> <thickness> <x'ref x> <x'ref y> <x'ref z>
...
```

If the standard shell finite element (without MITC) is needed, substitute `degshell` by `degshell_std`.

The stress resultants are defined as follows:

$$\begin{Bmatrix} N_{x'} \\ N_{y'} \\ N_{x'y'} \\ M_{x'} \\ M_{y'} \\ M_{x'y'} \\ V_{x'} \\ V_{x'} \end{Bmatrix} = \begin{Bmatrix} \left[\int_{-t/2}^{t/2}\int_{-\mathrm{d}y'/2}^{\mathrm{d}y'/2}\sigma_{x'x'}\,\mathrm{d}y'\mathrm{d}z'\right]/\mathrm{d}y' \\ \left[\int_{-t/2}^{t/2}\int_{-\mathrm{d}x'/2}^{\mathrm{d}x'/2}\sigma_{y'y'}\,\mathrm{d}x'\mathrm{d}z'\right]/\mathrm{d}x' \\ \left[\int_{-t/2}^{t/2}\int_{-\mathrm{d}y'/2}^{\mathrm{d}y'/2}\tau_{x'y'}\,\mathrm{d}y'\mathrm{d}z'\right]/\mathrm{d}y' \\ \left[\int_{-t/2}^{t/2}\int_{-\mathrm{d}y'/2}^{\mathrm{d}y'/2}(-z')\cdot\sigma_{x'x'}\,\mathrm{d}y'\mathrm{d}z'\right]/\mathrm{d}y' \\ \left[\int_{-t/2}^{t/2}\int_{-\mathrm{d}x'/2}^{\mathrm{d}x'/2}(-z')\cdot\sigma_{y'y'}\,\mathrm{d}x'\mathrm{d}z'\right]/\mathrm{d}x' \\ \left[\int_{-t/2}^{t/2}\int_{-\mathrm{d}y'/2}^{\mathrm{d}y'/2}(-z')\cdot\tau_{x'y'}\,\mathrm{d}y'\mathrm{d}z'\right]/\mathrm{d}y' \\ \left[\int_{-t/2}^{t/2}\int_{-\mathrm{d}y'/2}^{\mathrm{d}y'/2}\tau_{x'z'}\,\mathrm{d}y'\mathrm{d}z'\right]/\mathrm{d}y' \\ \left[\int_{-t/2}^{t/2}\int_{-\mathrm{d}x'/2}^{\mathrm{d}x'/2}\tau_{y'z'}\,\mathrm{d}x'\mathrm{d}z'\right]/\mathrm{d}x' \end{Bmatrix} = \int_{-t/2}^{t/2} \begin{Bmatrix} \sigma_{x'x'} \\ \sigma_{y'y'} \\ \tau_{x'y'} \\ -z'\cdot\sigma_{x'x'} \\ -z'\cdot\sigma_{y'y'} \\ -z'\cdot\tau_{x'y'} \\ \tau_{x'z'} \\ \tau_{y'z'} \end{Bmatrix} \mathrm{d}z' \qquad (2.19)$$

and they are shown in Figure 2.9.

2.4.3 Section `regions`

In a n-dimensional problem, a region (or subdomain) Ω is a n-dimensional entity that is a part or the whole domain of the problem. In MultiFEBE, a region Ω can be discretized by one of two different methods:

- **BEM (BE region).** The region is treated by the BEM. Its discretization is defined by its boundary $\partial\Omega$, which in general is built using a set of boundaries, which in turn is associated with mesh parts and hence with boundary elements.
- **FEM (FE region).** The region is treated by the FEM. Its discretization is defined by a set of FE subregions, which in turn is associated with parts and hence with finite elements. FE region can only be made of elastic solid materials, i.e. finite elements for fluid or poroelastic medium are not availabe.

The interaction (coupling) between BE regions is done through their shared boundaries, which must have opposite orientation for each region, see Figure 2.11. The interaction between FE regions is done through their shared nodes as usual. The interaction between BE boundaries and FE regions is done through the BE boundaries and the boundary of the FE elements that shares the same position. The interaction between BE and FE elements is automatically detected.

The format of this section is explained in the following. The first line indicates the number of regions. Then, for each region there must be a block of data consisting of several lines of data. The first one is the region identifier and the region class (discretization method: "fe" or "be"). If the region is a BE region, then the second line indicates the number of boundaries and a list of boundaries (with their orientation signs). If the region is a FE region, then the second line indicates the number of FE subregions and a list of fe subregions. The third line defines the material. Then, only if the region is a BE region, the fourth line defines the number and a list of BE body loads. Also, only if the region is a BE region and the analysis is time harmonic, the fifth line defines the number and a list of incident fields. The general format of the section is:

general format of section [regions]

```
[regions]
<n = number of regions>
```

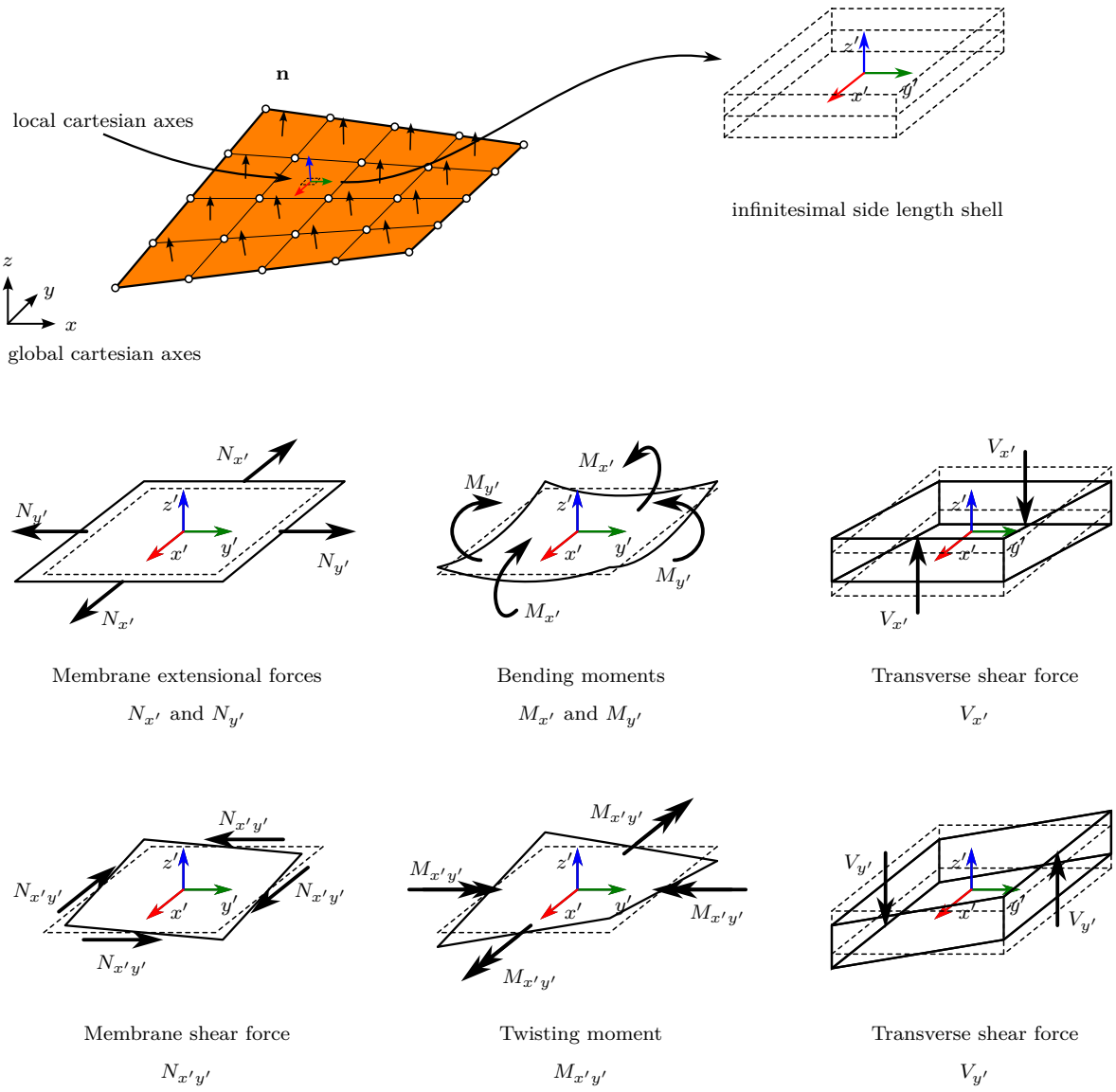


Figure 2.9: Shell stress resultants: $N_{x'}$, $N_{y'}$, $N_{x'y'}$, $M_{x'}$, $M_{y'}$, $M_{x'y'}$, $V_{x'}$, $V_{y'}$

```
<region 1 identifier> <region class (discretization method): be or fe>
<number of boundaries or fe subregions> <list of boundaries or fe subregions identifiers>
material <list of materials>
[<if be: number of be body loads> <list of be body loads>]
[<if be and analysis=harmonic: number of incident fields> <list of incident fields>]

...

<region n identifier> <region class (discretization method): be or fe>
<number of boundaries or fe subregions> <list of boundaries or fe subregions identifiers>
material <list of materials>
[<if be: number of be body loads> <list of be body loads>]
[<if be and analysis=harmonic: number of incident fields> <list of incident fields>]
```

2.4.4 Section `fe subregions`

A FE subregion is a partition of a FE region. A FE subregion is associated with a part of the mesh. The general format of the section is:

general format of section [fe subregions]

```
[fe subregions]
<n = number of FE subregions>
<FE subregion 1 identifier> <part identifier> 0 0
<FE subregion 2 identifier> <part identifier> 0 0
...
<FE subregion n identifier> <part identifier> 0 0
```

where the last two zeros at the end of the each line are mandatory, and they are going to be used in the future for additional features.

2.4.5 Section `boundaries`

In a n-dimensional problem, a boundary Γ is a $(n-1)$-dimensional oriented entity that is a part or the whole boundary of a region. The whole boundary $\partial\Omega$ of a region Ω treated by the Boundary Element Method, BE region in the following, is defined as a set of boundaries:

$$\partial\Omega = \{\ldots, \Gamma_j, \ldots\} \tag{2.20}$$

whose orientation must be outwards from the BE region. A minus sign before Γ_j can be used to indicate the reversion of the orientation of Γ_j in order to get a compatible $\partial\Omega$.

There are two main classes of boundaries:

- **Ordinary.** An ordinary boundary is a boundary in the classical sense. It can be connected with one or two BE regions. In both cases, the boundary can be connected with FE elements.

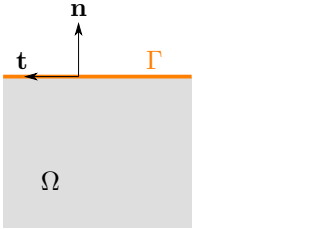


Figure 2.10: Ordinary boundary connected with one region

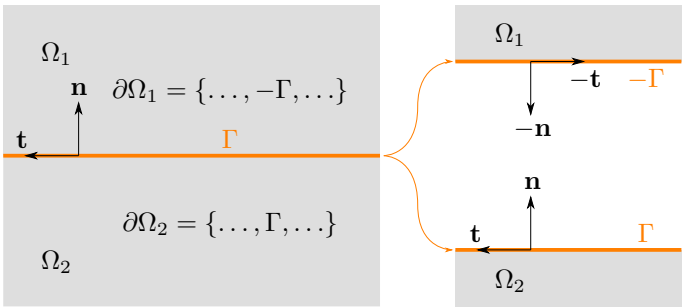


Figure 2.11: Ordinary boundary connected with two regions. Right: with exploded view.

- **Crack-like.** A special boundary that lies inside a BE region and is composed by two crack-like sub-boundaries (Γ^+ and Γ^-) of opposite orientations. Thus, a crack-like boundary can be connected only with one BE region. Generally speaking, it is the condensed geometric description of a null

thickness inclusion or void. Its orientation defines the orientation of the positive sub-boundary Γ^+, i.e. it defines which face is Γ^+ and which face is Γ^-.

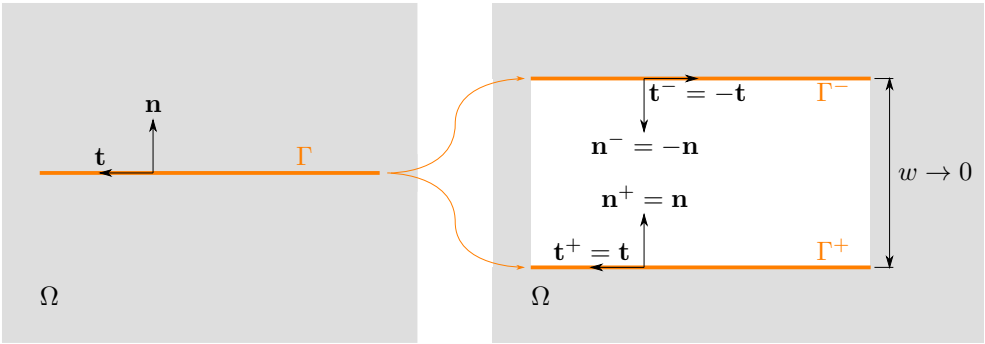


Figure 2.12: Crack-like boundary. Right: exploded view.

Each boundary is build up with unique boundary elements and nodes, which are defined by a "part" of the mesh. Thus, all boundaries must be connected with one and only one mesh part. The orientation of all boundary elements in that part must be the same, i.e. be compatible, which defines the orientation of the boundary.

The first line indicates the number of boundaries. Then, one line per boundary indicating the boundary identifier, the identifier of the part that discretize it, and finally the boundary class (`ordinary` or `crack-like`). The general format of the section is:

general format of section [boundaries]

```
[boundaries]
<n = number of boundaries>
<boundary 1 identifier> <part identifier> <boundary class: ordinary or crack-like>
<boundary 2 identifier> <part identifier> <boundary class: ordinary or crack-like>
...
<boundary n identifier> <part identifier> <boundary class: ordinary or crack-like>
```

Next, it is shown an example with 4 boundaries, where boundary 1 is the part 1 of the mesh and is an ordinary boundary, boundary 3 is the part 6 of the mesh and is an ordinary boundary, boundary 4 is the part 2 of the mesh and is a crack-like boundary, and boundary 2 is the part 3 of the mesh and is an ordinary boundary:

input.dat

```
...

[boundaries]
4
1 1 ordinary
3 6 ordinary
4 2 crack-like
2 3 ordinary

...
```

It is important to highlight that, in case of adjacent boundaries with different boundary conditions or with geometries such that a discontinuity in tractions will arrive, it is necessary to double (duplicate) the nodes in the boundaries of such BEM boundaries. A non-nodal collocation strategy is followed by default in all these boundary nodes.

2.4.6 Section `be bodyloads`

Body loads in BE regions are defined in this section. The general format follows a simpler pattern as the previous section. The first line contains the number of BE body loads to be defined, next as many lines are BE body loads. Each line contains first the BE body load identifier, and last the mesh part which contains the elements associated with it. The general format can be written as:

general format of section [fe subregions]

```
[be bodyloads]
<n = number of BE body loads>
<BE bodyload 1 identifier> <part identifier>
```

```
<BE bodyload 2 identifier> <part identifier>
...
<BE bodyload n identifier> <part identifier>
```

where the last two zeros at the end of the each line are mandatory, and they are going to be used in the future for additional features.

2.5 Auxiliary entities of the model

2.5.1 Internal points

Internal points are points within a BE region where field values (displacements, pressures, stresses, ...) are requested. They must not lie at any boundary. They are obtained at post-processing stage after boundary values are obtained.

There are two ways of requesting the calculation of fields at internal points:

- List of internal points defined by identifiers, region which contains each one, and point coordinates.

general format of section [internal points]

```
[internal points]
<# internal points>
<internal point id> <region id> <x coord.> <y coord> <z coord.>
...
```

- From the nodes defined in a mesh (see 2.2.3 for details about mesh file format):

general format of section [internal points from mesh]

```
[internal points from mesh]
mesh_file = <mesh file format> <path to mesh files>
region_association : <# selected mesh parts>
<selected mesh part id> <region id>
...
```

The program does not check if each internal point is indeed inside the specified region, so the user is responsible of the coherence between these data.

2.5.2 Internal elements

To be documented.

2.5.3 Groups

Groups are sets of entities of similar nature used to assign similar boundary conditions, settings or other attributes. There can be groups of nodes or groups of elements (but not mixed), and there are several ways to gather the nodes or elements of each group:

- Create a group containing all nodes or elements.
- Create a group containing all nodes or elements belonging of a given part.
- Create a group containing the nodes or elements indicated in a list.
- Create a group containing all nodes or elements in the interior or exterior of a bounding ball or box.

The format for the data section in each case is:

general format of section [groups]

```
[groups]
<n = number of groups>
<group id> nodes all
<group id> nodes part <part_id>
<group id> nodes list <N = number of nodes> <node 1> <node 2> ... <node N>
<group id> nodes ball <"interior" or "exterior"> <center> <radius> <id part>
<group id> nodes box <"interior" or "exterior"> <xmin> <xmax> <ymin> <ymax> [<zmin> <zmax>] <id part>
<group id> elements all
<group id> elements part <part_id>
<group id> elements list <N = number of elements> <element 1> <element 2> ... <element N>
```

Axes	Type	Name	Values	Equations
global	0	displacement	U	$u_k = U$
	1	traction	T	$t_k = T$
	4	infinitesimal rotation field	center $\mathbf{c}$, axis $\mathbf{a}$, angle θ	$u_k = \theta\,[\mathbf{a} \times (\mathbf{x} - \mathbf{c})] \cdot \mathbf{e}_k$
	10	normal pressure	P	$t_k = P n_k$
local	2	displacement	U	$\mathbf{u} \cdot \mathbf{l}_k = U$
	3	traction	T	$\mathbf{t} \cdot \mathbf{l}_k = T$

Note: $k = 1, \ldots, n$ (global or local axes). Local axes are $\{\mathbf{l}_1, \mathbf{l}_2, \mathbf{l}_3\} = \{\mathbf{n}, \mathbf{t}_1, \mathbf{t}_2\}$.

Table 2.10: Boundary conditions for ordinary boundaries

```
<group id> elements ball <"interior" or "exterior"> <center> <radius> <id part>
<group id> elements box <"interior" or "exterior"> <xmin> <xmax> <ymin> <ymax> [if 3D: <zmin> <zmax>] <id part>
...
```

2.6 Model conditions

In this section, the data sections related to the conditions (boundary and interface conditions, loads, symmetry planes) of the model are explained.

This part of the manual is incomplete as it only deals with conditions for elastic solids.

2.6.1 Section `symmetry planes`

If present, in this section the symmetry planes of the problem are defined. All symmetry planes are assumed to be at the origin of coordinates. By using symmetry planes, only a half, quarter or octant of the model has to be discretized. For BE regions, no additional boundary conditions are required since influence matrices are built taking symmetry planes into account. For FE regions, the corresponding boundary conditions are applied to nodes at symmetry planes.

In order to indicate the existence and the nature of the symmetry, it is necessary to define up to three of the symmetry planes implemented: plane with unit normal $\mathbf{n} = \mathbf{e}_1 = (1, 0, 0)$ (plane yz), plane with unit normal $\mathbf{n} = \mathbf{e}_2 = (0, 1, 0)$ (plane zx), or plane with unit normal $\mathbf{n} = \mathbf{e}_3 = (0, 0, 1)$ (plane xy); where each of one can be either of symmetry or antisymmetry. The general format for the section is:

general format of section [symmetry planes]

```
[symmetry planes]
<"plane_n1", "plane_n2" or "plane_n3"> : <"symmetry" or "antisymmetry">
...
```

For example, if the plane with normal $\mathbf{n} = \mathbf{e}_1 = (1, 0, 0)$ (plane yz) is a symmetry plane, i.e. fields at (x_1, x_2, x_3) are symmetric to fields at $(-x_1, x_2, x_3)$, then:

input.dat

```
...

[symmetry planes]
plane_n1: symmetry

...
```

2.6.2 Section `conditions over be boundaries`

In this section, the boundary conditions applied on the boundary elements boundaries are defined. All boundaries except the interfaces and the boundaries that are coupled with finite elements need to specify their boundary conditions. If not defined, a traction-free boundary is assumed by default. Here, boundary conditions for viscoelastic solids are explained.

There are available four boundary conditions for **ordinary boundaries**, two on global axes, and two in local axes, see Table 2.10. The two boundary conditions of each group are known displacement and known traction. For each boundary, the user must specify the boundary condition for each coordinate in local axes or in global axes, but not mixed.

Face	Type	Name	Values	Equations
Γ^+	0	displacement	U^+	$u_k^+ = U^+$
	1	traction	T^+	$t_k^+ = T^+$
Γ^-	0	displacement	U^-	$u_k^- = U^-$
	1	traction	T^-	$t_k^- = T^-$

Note: $k = 1, \ldots, n$ (global axes).

Table 2.11: Boundary conditions for crack-like boundaries

It is always preferable using the B.C. expressed in global axes (B.C. 0 or 1) when possible because they do not need additional equations. If the boundary is planar and its normal vector is parallel to any global axis, then the B.C. expressed in local axes are not needed at all.

There are available two boundary conditions for each face of a **crack-like boundary**. One establishes a displacement, while the other establishes a traction, see Table 2.11.

The general format of the section is:

general format of section [symmetry planes]

```
[conditions over be boundaries]
boundary <boundary id>: <type x> <value x>
                        <type y> <value y>
                        <type z> <value z>
                        <type x (if crack-like: face -)> <value x (if crack-like: face -)>
                        <type y (if crack-like: face -)> <value y (if crack-like: face -)>
                        <type z (if crack-like: face -)> <value z (if crack-like: face -)>
...
```

where note that for two-dimensional problems, the z lines must be removed. An example of a two-dimensional problem with two boundaries with identifiers `1` and `3`, being the first an ordinary boundary with null displacements and the second a crack-like boundary with traction-free faces:

input.dat

```
...

[conditions over be boundaries]
boundary 1: 0 0.
            0 0.
boundary 3: 1 0.
            1 0.
            1 0.
            1 0.

...
```

The same example as before but for a three-dimensional problem:

input.dat

```
...

[conditions over be boundaries]
boundary 1: 0 0.
            0 0.
            0 0.
boundary 3: 1 0.
            1 0.
            1 0.
            1 0.
            1 0.
            1 0.

...
```

Note that if the analysis is time harmonic, the values of the boundary conditions must be introduced as complex numbers, e.g.`(0.,0.)`.

2.6.3 Section `conditions over fe elements`

To be documented.

2.6.4 Section `conditions over nodes`

In this section, the boundary conditions over nodes are applied. For nodes of boundary elements, this boundary condition over nodes overrides the boundary conditions applied over their boundaries, and the Tables 2.10 and 2.11 remains valid. For nodes of finite elements, the boundary conditions are applied to the displacements and the rotation, the type of boundary conditions are `0` for known displacement, and `1` for known force (or moment).

The general format for the section is similar to the conditions over boundaries:

general format of section [conditions over nodes]

```
[conditions over nodes]
node <node id>: <type x> <value x>
                <type y> <value y>
                <type z> <value z>
                <if fe: type for rot. x or alpha> <value for rot. x or alpha>
                <if fe: type for rot. y or beta>  <value for rot. y or beta>
                <if fe: type for rot. z>          <value for rotation z>
```

For a two-dimensional problem, example of a clamped beam FE node with identifier `67`, and a pin beam FE node with identifier `41`:

input.dat

```
...

[conditions over nodes]
node 67: 0 0.
         0 0.
         0 0.
node 41: 0 0.
         0 0.
         1 0.
```

2.6.5 Section `incident waves`

In this section, the incoming waves are defined. The incidence angles are shown in Figure 2.13.

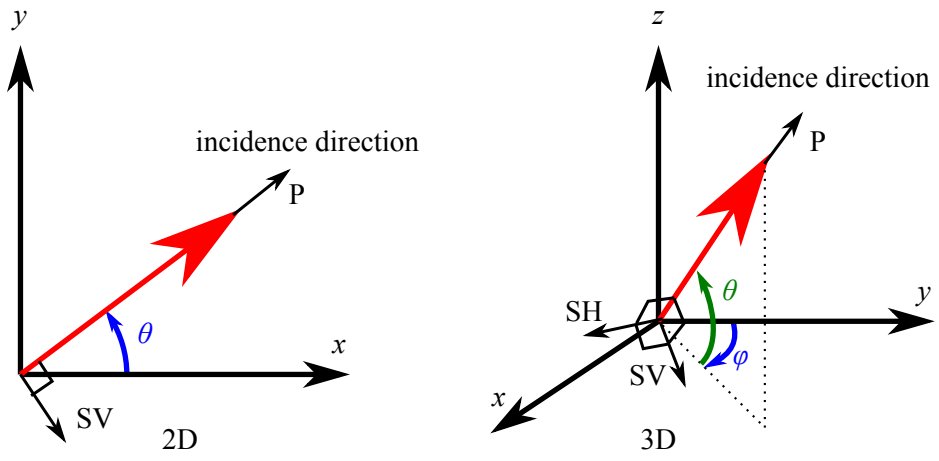


Figure 2.13: Incidence angles for two- and three-dimensional problems

The incident waves can be defined in terms of displacements (unitary displacements), in terms of stresses (unitary stresses) or in terms of the potentials (unitary potentials). The general format for the section is:

general format of section [incident waves]

```
[incident waves]
<number of incident waves>
```

```
<incident wave identifier>
<class: plane>
<space> [if space=half-space: <np> <xp> <bc>]
<variable> <amplitude> <x0(1)> <x0(2)> [if 3D: <x0(3)>] [if 3D: <varphi>] <theta>
<xs(1)> <xs(2)> [if 3D: <xs(3)>] <symconf(1)> <symconf(2)> [if 3D: <symconf(3)>]
<region_type> <wave_type>

...
```

where angles have to be introduced in degrees.

Example of vertical P-wave in terms of displacements in the full-space:

input.dat

```
...

[incident waves]
1
1
plane
full-space
0 1. 0. 0. 90.
0. 0. 0. 0.
viscoelastic p

...
```

Example of vertical SV-wave in terms of displacements in the half-space:

input.dat

```
...

[incident waves]
1
1
plane
half-space 2 0. 1
0 1. 0. 0. 90.
0. 0. 0. 0.
viscoelastic sv

...
```

Chapter 3

Output files

3.1 Introduction

Output files are written according to the type of analysis and model, as well as the input file export section. By default the path and name of the output files is equal to the input file but with an additional extension. The path and name can be changed when calling the solver from the terminal as explained in 1.5. There are three main native output files with the following additional extensions: nodal solutions (`*.nso`), element solutions (`*.eso`) and stress resultant solutions (`*.tot`). The specific file format is different for static and time harmonic analysis. There is one Gmsh output file (`*.pos`) which contains the case mesh and results (MSH file format version 2.2).

3.2 Nodal solutions file (`*.nso`)

This output file is a plain text file containing the nodal (node by node) results. The file starts with a set of header lines whose first character is "`#`" (a comment line in GNU/Linux environments), describing the file and the meaning of the main data columns. The first 9 columns are always present, and indicates the following:

Column	Value	Description
$1	integer	Analysis step index. For linear elastic static analysis it is always 0. For time harmonic analysis it is the frequency index. In future releases, it will receive the natural frequency index (for modal analysis), or the time step index (for transient analysis), or the loading step index (for non-linear static analysis).
$2	float	Analysis step value. For linear elastic static analysis it is always 0.0. For time harmonic analysis it is the frequency value (in Hz or rad/s depending on the units used in the frequencies section of the input file). In future releases, it will receive the natural frequency value (for modal analysis), or the time step value (for transient analysis), or the loading step value (for non-linear static analysis).
$3	integer	Identifier of the region to which the node result belongs. In the case of finite element nodes, this has no significance since results does not depend on the region. However, in the case of boundary elements belonging to two regions (interface boundary elements) this allows selecting which result is required.
$4	integer	Region class: 1 (BE region) or 2 (FE region).
$5	integer	Region type: 1 (inviscid fluid) or 2 (elastic solid) or 3 (poroelastic medium).

Column	Value	Description
$6	integer	Identifier of the BE boundary (if $4==1) or FE subregion (if $4==2) to which the node result belongs. In the case of finite element nodes, this has no significance since results does not depend on the FE subregion. However, in the case of boundary elements belonging to two regions (interface boundary elements) this allows selecting which result is required.
$7	integer	If $4==1, then it indicates the boundary class: 1 (ordinary) or 2 (crack-like). If $4==2, then it indicates the number of degrees of freedom of the node.
$8	integer	If $4==1, then it indicates the boundary face: 1 (positive normal) or 2 (negative normal). If $4==2, then it is always 0.
$9	integer	Identifier of the node.

From column $10 onwards, the meaning of each column differs depending on the problem dimension (2D or 3D), on the type of analysis (static or time harmonic), on the region class (BE or FE), on the region type (inviscid fluid, elastic solid, poroelastic medium), and on the number of DOF (if a FE region).

For 2D problems in a static analysis (all regions are only of the elastic solid type):

Column	Value	Description
$10,$11	float	Node (x_1, x_2) coordinates.
		BE region node
$12,$13	float	Node displacements (u_1, u_2).
$14,$15	float	Node tractions (t_1, t_2).
		FE region node with 2 DOFs
$12,$13	float	Node displacements (u_1, u_2).
$14,$15	float	Node forces (f_1, f_2).
		FE region node with 3 DOFs
$12-$14	float	Node displacements/rotation (u_1, u_2, θ_3).
$15-$17	float	Node forces/moment (f_1, f_2, m_3).

For 3D problems in a static analysis (all regions are only of the elastic solid type):

Column	Value	Description
$10-$12	float	Node (x_1, x_2, x_3) coordinates.
		BE region node
$13-$15	float	Node displacements (u_1, u_2, u_3).
$16-$18	float	Node tractions (t_1, t_2, t_3).
		FE region node with 3 DOFs
$13-$15	float	Node displacements (u_1, u_2, u_3).
$16-$18	float	Node forces (f_1, f_2, f_3).
		FE region node with 5 DOFs (shell element nodes with local rotations)
$13-$17	float	Node displacements/local rotations $(u_1, u_2, u_3, \alpha, \beta)$.
$18-$22	float	Node forces/local moments $(f_1, f_2, f_3, m_\alpha, m_\beta)$.

Column	Value	Description
		FE region node with 6 DOFs
\$13-\$18	float	Node displacements/rotations $(u_1, u_2, u_3, \theta_1, \theta_2, \theta_3)$.
\$19-\$24	float	Node forces/moments $(f_1, f_2, f_3, m_1, m_2, m_3)$.

For 2D problems in a time harmonic analysis, where each node variable is written by two consecutive columns containing its real and imaginary parts (if `complex_notation = cartesian` in `[export]` section) or absolute value and argument (if `complex_notation = polar` in `[export]` section):

Column	Value	Description
\$10,\$11	float	Node (x_1, x_2) coordinates.
		BE region node (inviscid fluid)
\$12,\$13	float	Node pressure p (total field).
\$14,\$15	float	Node normal displacement U_n (total field).
\$16,\$17	float	Node pressure p^{inc} (incident field).
\$18,\$19	float	Node normal displacement U_n^{inc} (incident field).
\$20-\$23	float	Node displacement (U_1, U_2) (total field).
		BE region node (elastic solid)
\$12-\$15	float	Node displacements (u_1, u_2) (total field).
\$16-\$19	float	Node tractions (t_1, t_2) (total field).
\$20-\$23	float	Node displacements $(u_1^{\text{inc}}, u_2^{\text{inc}})$ (incident field).
\$24-\$27	float	Node tractions $(t_1^{\text{inc}}, t_2^{\text{inc}})$ (incident field).
		BE region node (poroelastic medium)
\$12,\$13	float	Node fluid equivalent pressure τ (total field). Note that it is related to the dynamic pore pressure by $\tau = -\phi p$, where ϕ is the porosity.
\$14-\$17	float	Node solid displacements (u_1, u_2) (total field).
\$18,\$19	float	Node fluid normal displacement U_n (total field).
\$20-\$23	float	Node solid tractions (t_1, t_2) (total field).
\$24,\$25	float	Node fluid equivalent pressure τ^{inc} (incident field).
\$26-\$29	float	Node solid displacements $(u_1^{\text{inc}}, u_2^{\text{inc}})$ (incident field).
\$30,\$31	float	Node fluid normal displacement U_n^{inc} (incident field).
\$32-\$35	float	Node solid tractions $(t_1^{\text{inc}}, t_2^{\text{inc}})$ (incident field).
\$36-\$39	float	Node fluid displacement (U_1, U_2) (total field).
		FE region node (only of elastic solid type) with 2 DOFs
\$12-\$15	float	Node displacements (u_1, u_2).
\$16-\$19	float	Node forces (f_1, f_2).
		FE region node (only of elastic solid type) with 3 DOFs
\$12-\$17	float	Node displacements/rotation (u_1, u_2, θ_3).
\$18-\$23	float	Node forces/moment (f_1, f_2, m_3).

For 3D problems in a time harmonic analysis, where as in the 2D case each node variable is written by two consecutive columns containing its real and imaginary parts (if `complex_notation = cartesian` in

`[export]` section) or absolute value and argument (if `complex_notation = polar` in `[export]` section):

Column	Value	Description
\$10-\$12	float	Node (x_1, x_2, x_3) coordinates.
		BE region node (inviscid fluid)
\$13,\$14	float	Node pressure p (total field).
\$15,\$16	float	Node normal displacement U_n (total field).
\$17,\$18	float	Node pressure p^{inc} (incident field).
\$19,\$20	float	Node normal displacement U_n^{inc} (incident field).
\$21-\$24	float	Node displacement (U_1, U_2, U_3) (total field).
		BE region node (elastic solid)
\$13-\$18	float	Node displacements (u_1, u_2, u_3) (total field).
\$19-\$24	float	Node tractions (t_1, t_2, t_3) (total field).
\$25-\$30	float	Node displacements $(u_1^{\text{inc}}, u_2^{\text{inc}}, u_3^{\text{inc}})$ (incident field).
\$31-\$36	float	Node tractions $(t_1^{\text{inc}}, t_2^{\text{inc}}, t_3^{\text{inc}})$ (incident field).
		BE region node (poroelastic medium)
\$13,\$14	float	Node fluid equivalent pressure τ (total field). Note that it is related to the dynamic pore pressure by $\tau = -\phi p$, where ϕ is the porosity.
\$15-\$20	float	Node solid displacements (u_1, u_2, u_3) (total field).
\$21,\$22	float	Node fluid normal displacement U_n (total field).
\$23-\$28	float	Node solid tractions (t_1, t_2, t_3) (total field).
\$29,\$30	float	Node fluid equivalent pressure τ^{inc} (incident field).
\$31-\$36	float	Node solid displacements $(u_1^{\text{inc}}, u_2^{\text{inc}}, u_3^{\text{inc}})$ (incident field).
\$37,\$38	float	Node fluid normal displacement U_n^{inc} (incident field).
\$39-\$44	float	Node solid tractions $(t_1^{\text{inc}}, t_2^{\text{inc}}, t_3^{\text{inc}})$ (incident field).
\$45-\$50	float	Node fluid displacement (U_1, U_2, U_3) (total field).
		FE region node (only of elastic solid type) with 3 DOFs
\$13-\$18	float	Node displacements (u_1, u_2, u_3).
\$19-\$24	float	Node forces (f_1, f_2, f_3).
		FE region node (only of elastic solid type) with 5 DOFs (shell element nodes with local rotations)
\$13-\$22	float	Node displacements/local rotations $(u_1, u_2, u_3, \alpha, \beta)$.
\$23-\$32	float	Node forces/local moments $(f_1, f_2, f_3, m_\alpha, m_\beta)$.
		FE region node (only of elastic solid type) with 6 DOFs
\$13-\$24	float	Node displacements/rotation $(u_1, u_2, u_3, \theta_1, \theta_2, \theta_3)$.
\$25-\$36	float	Node forces/moment $(f_1, f_2, f_3, m_1, m_2, m_3)$.

3.3 Element solutions file (`*.eso`)

This output file is a plain text file containing the element (element node by element node) results (stress resultants on finite elements). In a future release, element by element stress resultants over each boundary element will also be included. The file starts with a set of header lines whose first character is "`#`" (a

comment line in GNU/Linux environments), describing the file and the meaning of the main data columns. The first 13 columns are always present, and indicates the following:

Column	Value	Description
$1	integer	Analysis step index. For linear elastic static analysis it is always 0. For time harmonic analysis it is the frequency index. In future releases, it will receive the natural frequency index (for modal analysis), or the time step index (for transient analysis), or the loading step index (for non-linear static analysis).
$2	float	Analysis step value. For linear elastic static analysis it is always 0.0. For time harmonic analysis it is the frequency value (in Hz or rad/s depending on the units used in the frequencies section of the input file). In future releases, it will receive the natural frequency value (for modal analysis), or the time step value (for transient analysis), or the loading step value (for non-linear static analysis).
$3	integer	Identifier of the region to which the element node result belongs. In the case of finite element nodes, this has no significance since results does not depend on the region. However, in the case of boundary elements belonging to two regions (interface boundary elements) this allows selecting which result is required.
$4	integer	Region class: 1 (BE region) or 2 (FE region).
$5	integer	Region type: 1 (inviscid fluid) or 2 (elastic solid) or 3 (poroelastic medium).
$6	integer	Identifier of the BE boundary (if $4==1) or FE subregion (if $4==2) to which the element result belongs. In the case of boundary elements belonging to two regions (interface boundary elements) this allows selecting which result is required.
$7	integer	If $4==1, then it indicates the boundary class: 1 (ordinary) or 2 (crack-like). If $4==2, then it indicates the finite element dimension.
$8	integer	If $4==1, then it indicates the boundary face: 1 (positive normal) or 2 (negative normal). If $4==2, then it indicates the finite element type.
$9-$11	integer	Element identifier, dimension and type.
$12,$13	integer	Element node index and number of degrees of freedom.
$14	integer	Node identifier.

For 2D problems in a static analysis:

Column	Value	Description
$15,$16	float	Node (x_1, x_2) coordinates.
		Straight beam finite element
$17-$19	float	Axial force ($N_{x'}$), shear force ($V_{y'}$) and bending moment ($M_{z'}$).
		Bar finite element
$17	float	Axial force ($N_{x'}$).
		Discrete translational spring finite element
$17,$18	float	Force on each spring: $N_{x'}$, $N_{y'}$.
		Discrete translational/rotational spring finite element
$17-$19	float	Force/moment on each spring: $N_{x'}$, $N_{y'}$, $M_{z'}$.

For 3D problems in a static analysis:

Column	Value	Description
\$15-\$17	float	Node (x_1, x_2, x_3) coordinates.
		Straight beam finite element
\$18-\$23	float	Axial force ($N_{x'}$), shear forces ($V_{y'}$, $V_{z'}$), torsional moment ($M_{z'}$) and bending moments ($M_{y'}$, $M_{z'}$).
		Bar finite element
\$18	float	Axial force ($N_{x'}$).
		Discrete translational spring finite element
\$18-\$20	float	Force on each spring: $N_{x'}$, $N_{y'}$, $N_{z'}$.
		Discrete translational/rotational spring finite element
\$18-\$23	float	Force/moment on each spring: $N_{x'}$, $N_{y'}$, $N_{z'}$, $M_{x'}$, $M_{y'}$, $M_{z'}$.
		Shell finite element
\$18-\$25	float	Membrane forces ($N_{x'}$, $N_{y'}$, $N_{x'y'}$), bending moments ($M_{x'}$, $M_{y'}$), twisting moment ($M_{x'y'}$) and shear forces ($V_{x'}$, $V_{y'}$)).

For 2D problems in a time harmonic analysis, where each node variable is written by two consecutive columns containing its real and imaginary parts (if `complex_notation = cartesian` in `[export]` section) or absolute value and argument (if `complex_notation = polar` in `[export]` section):

Column	Value	Description
\$15,\$16	float	Node (x_1, x_2) coordinates.
		Straight beam finite element
\$17-\$22	float	Axial force ($N_{x'}$), shear force ($V_{y'}$) and bending moment ($M_{z'}$).
		Bar finite element
\$17,\$18	float	Axial force ($N_{x'}$).
		Discrete translational spring finite element
\$17-\$20	float	Force on each spring: $N_{x'}$, $N_{y'}$.
		Discrete translational/rotational spring finite element
\$17-\$22	float	Force/moment on each spring: $N_{x'}$, $N_{y'}$, $M_{z'}$.

For 3D problems in a time harmonic analysis, where each node variable is written by two consecutive columns containing its real and imaginary parts (if `complex_notation = cartesian` in `[export]` section) or absolute value and argument (if `complex_notation = polar` in `[export]` section):

Column	Value	Description
\$15-\$17	float	Node (x_1, x_2, x_3) coordinates.
		Straight beam finite element
\$18-\$29	float	Axial force ($N_{x'}$), shear forces ($V_{y'}$, $V_{z'}$), torsional moment ($M_{z'}$) and bending moments ($M_{y'}$, $M_{z'}$).
		Bar finite element

Column	Value	Description
$18,$19	float	Axial force ($N_{x'}$).
		Discrete translational spring finite element
$18-$23	float	Force on each spring: $N_{x'}$, $N_{y'}$, $N_{z'}$.
		Discrete translational/rotational spring finite element
$18-$29	float	Force/moment on each spring: $N_{x'}$, $N_{y'}$, $N_{z'}$, $M_{x'}$, $M_{y'}$, $M_{z'}$.
		Shell finite element
$18-$33	float	Membrane forces ($N_{x'}$, $N_{y'}$, $N_{x'y'}$), bending moments ($M_{x'}$, $M_{y'}$), twisting moment ($M_{x'y'}$) and shear forces ($V_{x'}$, $V_{y'}$).

The definition of the local cartesian basis and the stress resultants sign criteria for beams and shells are established according to Section 2.4.2 (see Figures 2.1, 2.2, 2.8 and 2.9).

3.4 Gmsh results file (*.pos)

The Gmsh output file (*.pos) contains the case mesh and results (MSH file format version 2.2). It is a plain text containing all this data which can be read by Gmsh. The following set of results are exported to Gmsh:

- Fluid pressure (positive faces). Only boundary elements.
- Fluid pressure (negative faces). Only boundary elements.
- Solid total displacements (positive faces). Boundary elements and finite elements.
- Solid total displacements (negative faces). Only boundary elements.
- Solid total tractions (positive faces). Only boundary elements.
- Solid total tractions (negative faces). Only boundary elements.
- FE nodal forces/reactions. Only finite elements.
- Beam stress resultants. Only finite elements.
- Beam stress resultants local axes x', y' and z'. Only finite elements.
- Bar stress resultants. Only finite elements.
- Shell stress resultants. Only finite elements.
- Shell stress resultants local axes x', y' and z'. Only finite elements.

Appendix A

How to compile the source code

Go to MultiFEBE's GitHub webpage, and download the source code. Then, decompress the `*.tar.gz` or `*.zip` file.

A.1 GNU/Linux

The steps to compile the source code are the usual on GNU/Linux projects using CMake. The particular commands shown below are for Debian distributions (e.g. Ubuntu), but it should be very similar for others.

1. First, you have to install the pre-requisites if not already installed, so you have to open a terminal and execute:

   - Install GNU Fortran, GNU Make and CMake:

     ```
     $ sudo apt-get install gfortran make cmake cmake-extras
     ```

   - Install OpenBLAS:

     ```
     $ sudo apt-get install libopenblas-base libopenblas-dev
     ```

2. Once you have all the pre-requisites, navigate at the source code main folder. Then create a temporary folder (e.g. `build`) where making the compilation, and navigate inside it:

   ```
   $ mkdir -p build
   $ cd build
   ```

3. Execute cmake taking into account that the CMake configuration file is in the parent folder. The basic command is:

   ```
   $ cmake -G "Unix Makefiles" ..
   ```

   If you want to compile an executable which uses OpenMP (shared-memory parallelization) and uses all possible compiler optimizations, you should use the Release build:

   ```
   $ cmake -D CMAKE_BUILD_TYPE=Release -G "Unix Makefiles" ..
   ```

   If you want to compile for debugging, you should use the Debug build:

   ```
   $ cmake -D CMAKE_BUILD_TYPE=Debug -G "Unix Makefiles" ..
   ```

   During the process, if any tool or dependency is missing, then CMake is going to notify you.

4. Once CMake correctly finishes, you can now execute GNU Make:

   ```
   $ make
   ```

5. If all this process ends correctly, you will have the executable `multifebe` there. However, it is still not available system-wide in the terminal. You have at this point several options:

(a) The simpler one is to copy the binary to `usr/bin`:

```
$ sudo cp multifebe /usr/bin/.
```

(b) The cleaner one is making a `*.deb` installer which will manage not only this, but it will also copy the documentation to the corresponding folders (`/usr/share/doc/multifebe`), and create a package registry so that it can also be completely uninstalled or updated in the future. In order to do so, you have to execute CPack as:

```
$ cpack
```

At the end of the process, you will have at your disposal your own `*.deb` installer. Then, you can follow the step described above in section 1.3.1.

A.2 Windows

The compilation in Windows requires much more steps than in GNU/Linux, but basically because it is required to install and to configure MSYS2. MSYS2 is a collection of tools and libraries providing you with an easy-to-use environment for building, installing and running native Windows software. Furthermore, it is very similar to GNU/Linux environments, so most of the programming work is shared when building for GNU/Linux and Windows.

1. Download and install MSYS2 from the webpage.
2. After the installation, the "MSYS2 MSYS" terminal is automatically opened. Close it.
3. We recommend to follow the post-installation instructions given at the webpage.
4. MSYS2 installs the so called Mintty terminal, but it provides separate launchers for different environments, such as the already opened "MSYS2 MSYS", but also "MinGW x64", which is the one used for compiling, and others. See Figure A.1.
5. We will reproduce the post-installation instructions given by MSYS2 at the date of this writing. First, open the "MSYS2 MSYS" terminal. See Figure A.1.

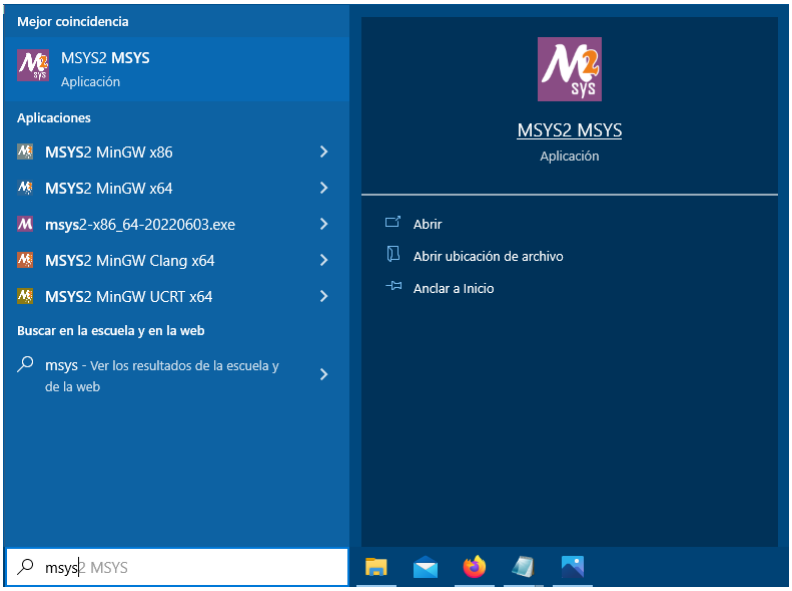


Figure A.1: Compilation in Windows: Steps 4 and 5

6. Update the packages by typing and executing the command `pacman -Syu` as shown in Figure A.3.
7. Proceed with the installation by typing `Y` and executing as shown in Figure A.3.
8. After updating, the terminal ask permission for closing. Type `Y` and execute.
9. Open again the "MSYS2 MSYS" terminal, and type and execute again the update command `pacman -Syu`.

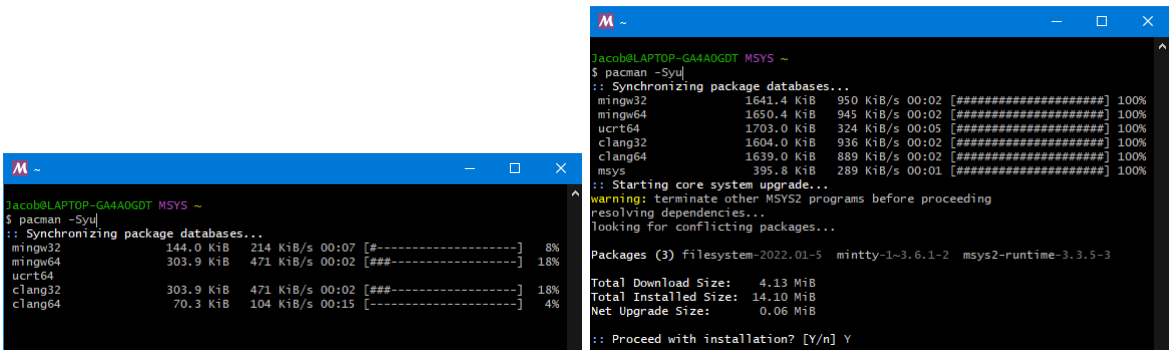


Figure A.2: Compilation in Windows: Steps 5 and 6

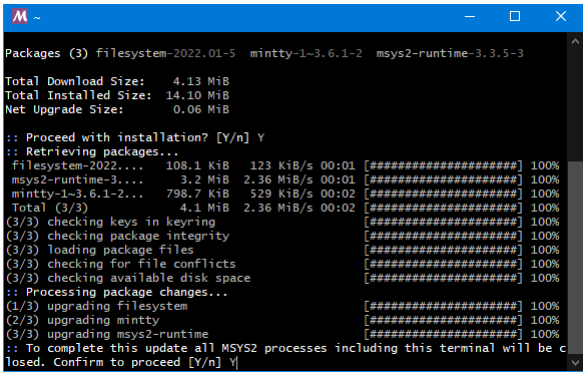


Figure A.3: Compilation in Windows: Step 8

10. Proceed with the installation by typing and executing `Y`.
11. Install typical tools for compiling by typing and executing:

    ```
    $ pacman -S mingw-w64-x86_64-toolchain
    ```

    which will take some time.
12. Close the "MSYS2 MSYS" terminal.
13. Open the "MSYS2 MinGW x64" terminal.
14. Install additional tools and libraries required by typing and executing the following sequence of commands:
    (a) Install GNU C Compiler and GNU Fortran for MinGW 64 bits:

    ```
    $ pacman -S gcc mingw-w64-x86_64-gcc gcc-fortran mingw-w64-x86_64-gcc-fortran
    ```

    (b) Install GNU Make, CMake and NSIS:

    ```
    $ pacman -S gcc make mingw-w64-x86_64-make cmake mingw-w64-x86_64-cmake mingw-w64-x86_64-nsis
    ```

    (c) Install OpenBLAS library:

    ```
    $ pacman -S gcc mingw-w64-x86_64-openblas
    ```
15. In the "MSYS2 MinGW x64" terminal, navigate to the source code main folder. In the case illustrated in the following figures `D:\multifebe`.
16. Create a folder where all compilation build files are generated. In the case illustrated in the following figures it is named `build`. This can be done by executing:

    ```
    $ mkdir -p build
    ```
17. Navigate to it by executing:

Figure A.4: Compilation in Windows: Step 13

```
$ cd build
```

18. Execute `cmake` taking into account that we want to use make as the compilation tool (thus using the option `-G ``Unix Makefiles''`) and that the configuration file `CMakeLists.txt` is in the parent folder:

```
$ cmake -G "Unix Makefiles" ..
```

If you want to compile an executable which uses OpenMP (shared-memory parallelization) and uses all possible compiler optimizations, you should use the Release build:

```
    $ cmake -D CMAKE_BUILD_TYPE=Release -G "Unix Makefiles" ..
```

If you want to compile for debugging, you should use the Debug build:

```
    $ cmake -D CMAKE_BUILD_TYPE=Debug -G "Unix Makefiles" ..
```

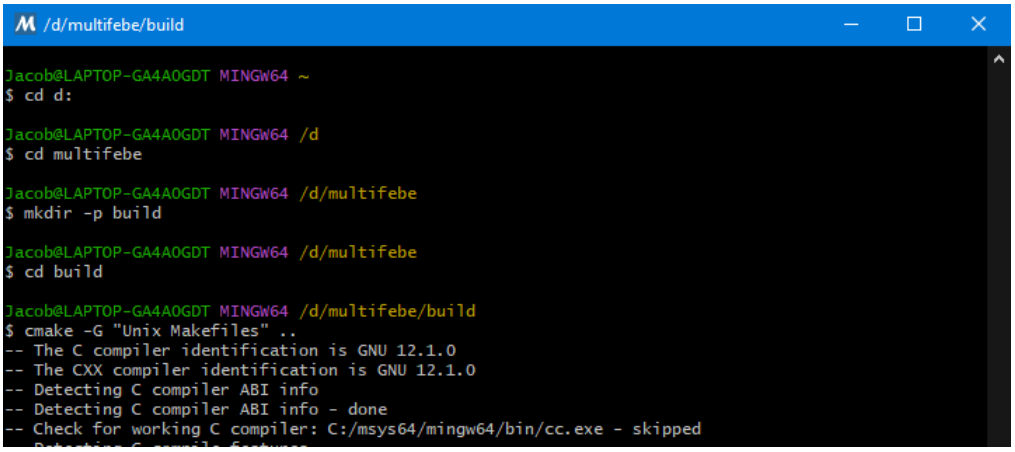


Figure A.5: Compilation in Windows: Steps 15 to 18

19. Execute `make` for compiling the source code:

```
$ make
```

20. After this is correctly done, `multifebe.exe` should be ready to be used in the current folder.

21. You can manually install this program in the system by creating a new folder inside `C:\Files (x86)` called for example `multifebe`, and copy `multifebe.exe` inside it. Then you have to add this directory to the system or user `PATH` environment variable. To do so, follow the instructions in this link, or do a search on the web about "How to set the path and environment variables in Windows". After this is done, you can run the program on a system terminal, like cmd or PowerShell.

22. It is also possible to build an installer which do all this for you, and furthermore integrates the program into your system with a uninstaller, etc. You have to be in a "MSYS2 MinGW x64" terminal, and the current working directory in the folder where we are compilating.

23. Then you can build the installer by executing in the terminal:

```
$ cpack
```

24. This will create and installer called `multifebe-x.x.x-w64.exe`, which is the installer we release for Windows. Then you can execute it and follow the instructions above in section **??**.

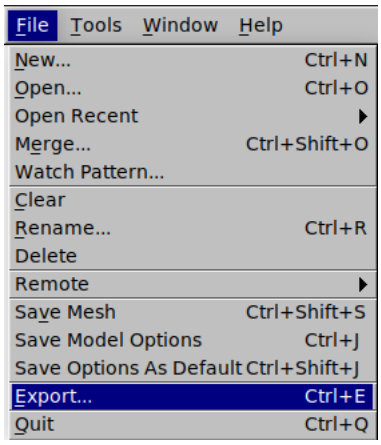
Appendix B

Gmsh - How to export a mesh in MSH file format version 2.2

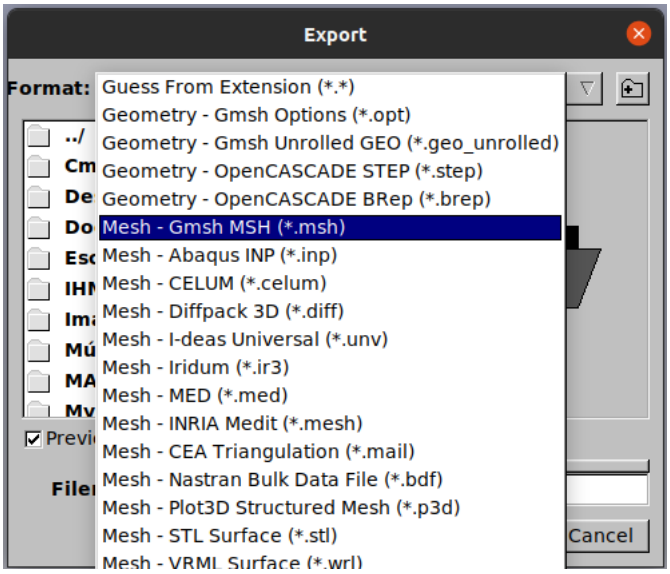
In Gmsh newer than version 3.0.6, the default Gmsh mesh file format is not MSH file format version 2.2. Therefore, you will have to save the mesh from an alternative workflow.

When using the Gmsh GUI, you have to follow the steps shown in Fig. B.1. When using the command-line interface, you have to include the option `-format msh2`, for example:

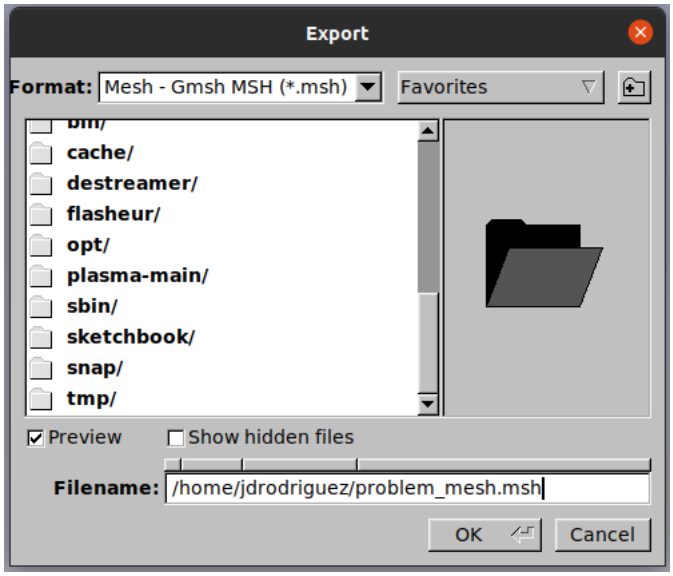
```
$ gmsh -2 -format msh2 pilecap.geo
```



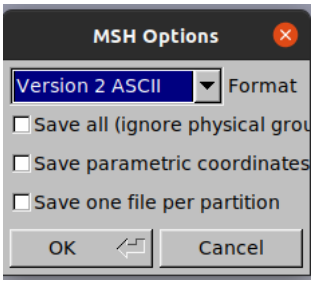
(a) Step 1: File > Export



(b) Step 2: Format: Mesh - Gmsh MSH (*.msh)



(c) Step 3: Write filename



(d) Step 4: Select MSH format

Figure B.1: Gmsh GUI: export mesh file in different format

Appendix C

GiD `*.bas` template file

We have implemented a `*.bas` template file for writing mesh files in the MultiFEBE native format from GiD pre- and post-processor. It is a simple plain text file containing a script in GiD language. Two versions of the template file are included in the package, one with carriage return of the Windows type (`multifebe_win.bas`), and another with the Unix type (`multifebe_unix.bas`). Note that the shown blank lines are necessary. Conceptually, each GiD "layer" is a "part" in MultiFEBE jargon.

`multifebe_win.bas & multifebe_unix.bas`

```

*IntFormat "%12i"
*RealFormat "%25.16e"
[parts]
*Set var nLayer=0
*loop layers
*Set var nLayer=LayerNum
*end layers
*nLayer
*loop layers
*LayerNum *LayerName
*end layers
[nodes]
*nPoin
*set elems(all)
*loop nodes
*NodesNum *NodesCoord
*end nodes
[elements]
*set Elems(All)
*nElem
*set Elems(Linear)
*loop elems
*if(ElemsNnode==2)
*ElemsNum line2 1 *ElemsLayerNum *ElemsConec
*endif
*if(ElemsNnode==3)
*ElemsNum line3 1 *ElemsLayerNum *ElemsConec
*endif
*end elems
*set Elems(Triangle)
*loop elems
*if(ElemsNnode==3)
*ElemsNum tri3 1 *ElemsLayerNum *ElemsConec
*endif
*if(ElemsNnode==6)
*ElemsNum tri6 1 *ElemsLayerNum *ElemsConec
*endif
*end elems
*set Elems(Quadrilateral)
*loop elems
*if(ElemsNnode==4)
*ElemsNum quad4 1 *ElemsLayerNum *ElemsConec
*endif
```

```
*if(ElemsNnode==8)
*ElemsNum quad8 1 *ElemsLayerNum *ElemsConec
*endif
*if(ElemsNnode==9)
*ElemsNum quad9 1 *ElemsLayerNum *ElemsConec
*endif
*end elems

```